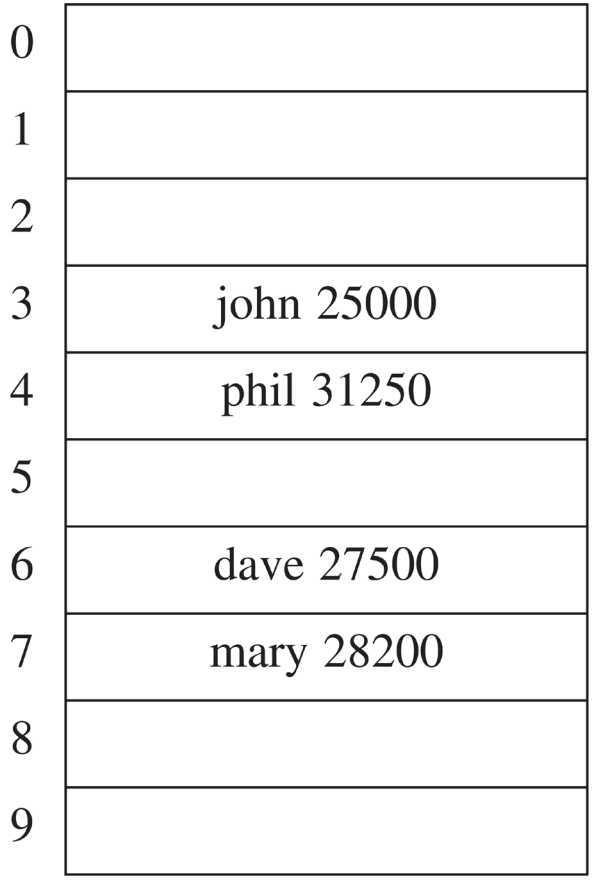
Advanced Hashing Techniques

Hashing

- Hashing is an algorithm (via a hash function) that maps large data sets of variable length, called keys, to smaller data sets of a fixed length
- Hashing is a technique used for performing insertions, deletions, and finds in constant average time
- A hash table (or hash map) is a data structure that uses a hash function to efficiently map keys to values, for efficient search and retrieval
- Widely used in many kinds of computer software, particularly for associative arrays, database indexing, caches, and sets

Idea

- Hash table data structure is an array of some fixed size containing the items
  - Each key is mapped into some number in the range 0 to TableSize − 1 and placed in the appropriate cell. The mapping is called a hash function
- Ideally the hash function should be simple to compute and should ensure that any two distinct keys get different cells
  - Since there are a finite number of cells and a virtually inexhaustible supply of keys, this is clearly impossible, and thus we seek a hash function that distributes the keys evenly among the cells



Maps

- The main idea at the heart of the hash table data structure is that it allows users to assign keys to elements and then use those keys later to look up or remove elements
  - This functionality defines a data structure known as a dictionary or map

- A map stores a collection of key-value pairs, (k, v), which we call items, where k is a key and v is a value that is associated with that key
  - In a map storing student records (such as the student's name, address, and course grades), the key might be the student's ID number

Maps

A map data structure, M, supports the following fundamental methods:

get(k): If M contains an item with key equal to k, then return the value of such an item; else return a special element NULL

put(k, v): Insert an item with key k and value v; if there is already an item with key k, then replace its value with v

remove(k): Remove from M an item with key equal to k, and return this item. If M has no such item, then return the special element NULL

NULL is called the sentinel. There are other methods such as size(), isEmpty() etc.

Lookup Tables

- In some cases, the universe of keys that will be used for a map is the set of integers in the range [0, N − 1], for a reasonably small value of N
  - There is an almost trivial way of implementing a map—namely, we can allocate an array, A, of size N, where each cell of A is thought of as a “bucket” that can hold a single key-element pairs (or a pointer to such a pair).
- Since we assume that keys are distinct, we can use A to store any key-value pair (k, v) by placing that pair in the cell A[k]. In this case, we refer to this implementation as a lookup table, since the key k allows us to simply “look up” the item for k by a single array-indexing operation

Lookup Tables

Performing the essential operations for a map in this case is quite simple. We begin by allocating the array A so that every cell is initially associated with the special NULL object. Then, we perform the map operations as follows:

- To perform a put(k, v) operation, we assign (k, v) to A[k]
- To perform a get(k) operation, we return A[k]
- To perform a remove(k) operation, we return A[k] and then we assign the NULL item to A[k]

Lookup Tables

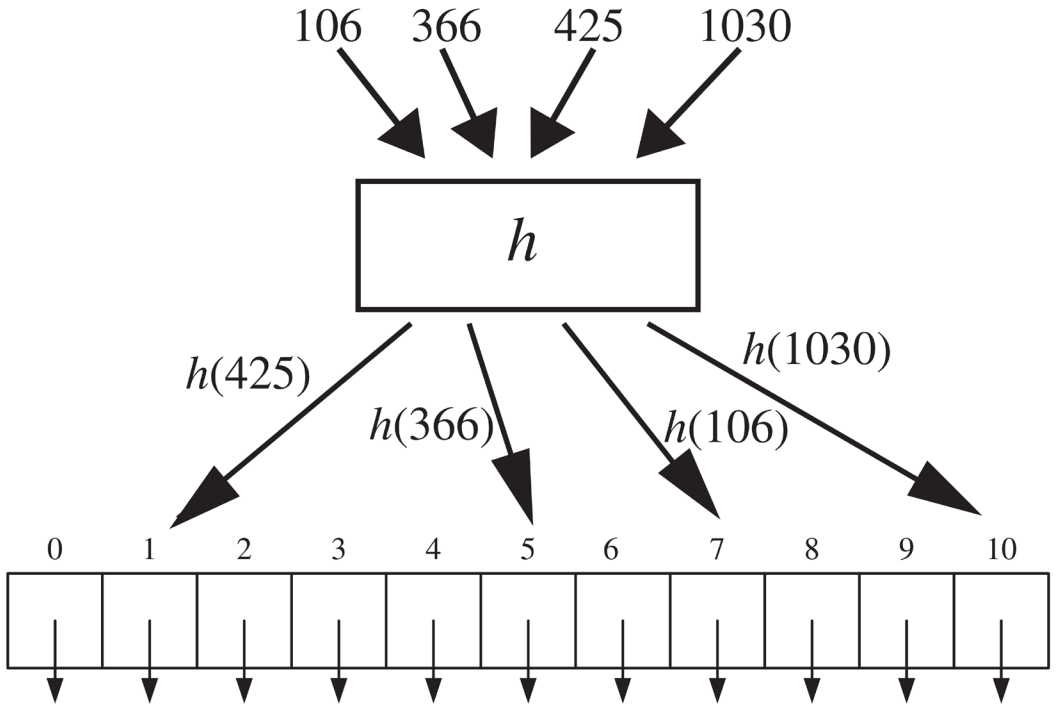
- Each of the essential map methods runs in O(1) time in worst case
- In terms of space usage, we note that the total amount of memory used for a lookup table is O(N)
- Another drawback with this implementation is that it requires keys be unique integers in the range [0, N − 1], which is often not the case

Hash Functions

- If we cannot assume that keys are unique integers in the range [0, N − 1], but we nevertheless still want to use something akin to a lookup table for storing key-value pairs, then we need a good way of assigning keys to integers in this range

- That is, we need a function, h, called a hash function, that maps each key k in our map to an integer in the range [0, N − 1], where N is the capacity of the underlying array for this table

- The use of such a function allows us to treat objects, such as strings, as numbers



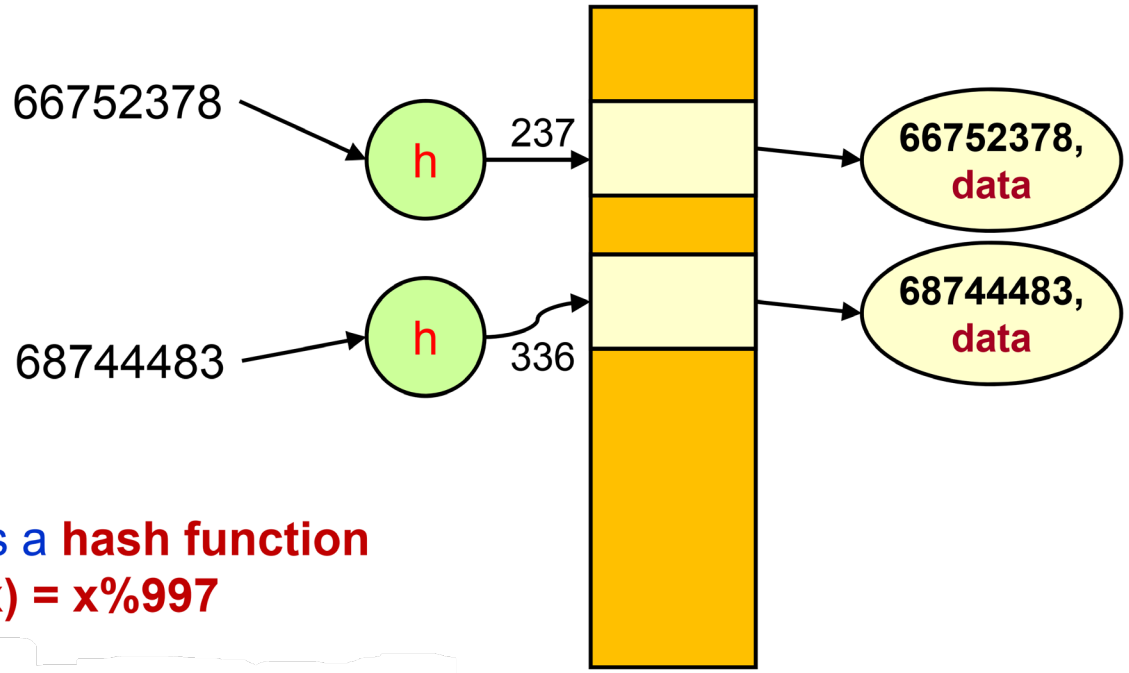
Collisions

- One complication that we have to deal with when using this approach, however, is that it is entirely possible that the hash function, h, could map two distinct keys to the same location in A
  - That is, we could have h(k) = h(l), for k = l, for two keys k and l in our map. We refer to such a pair of keys with the same hash value, j, as causing a collision at j

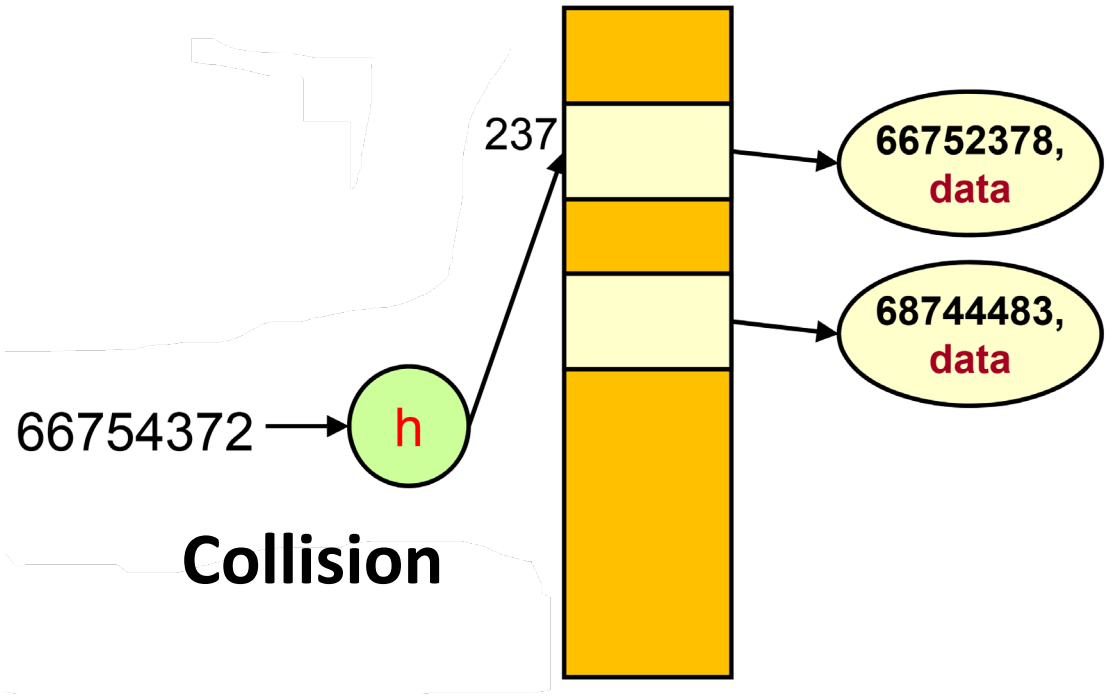
- We would like the hash function h to look as random as possible, since a truly random function would automatically minimize the expected number of collisions for any given hash value, j

- For practical reasons, we also would like the evaluation of a given hash function to be fast and easy to compute.

Example: Phone Number



h is a **hash function**
h(x) = x%997



Hashing of Strings

Simple Idea

```
hash(s) {
    sum = 0
    for each character c in s {
        sum += c
    }
    return sum % m
}
```

Problem:

Hash value is independent of the positions of the characters

e.g. cat=act, inch=chin etc.

Polynomial String Hashing

- Factoring in the positional information of the components in a key $k = (x_1, x_2, \ldots, x_d)$, is to choose a nonzero constant a and use as a hash function the following:

$$h(k) = x_1 a^{d-1} + x_2 a^{d-2} + \cdots + x_{d-1} a + x_d$$

- We can concert the word “ace” into a number:
  - key = $1 * 26^2 + 3 * 26^1 + 5 * 26^0$

```
function hash(key, arraySize) {
  const base = 13;

  let hash = 0;
  for (let charIndex = 0; charIndex < key.length;
charIndex += 1) {
    const charCode = key.charCodeAt(charIndex);
    hash += charCode * (base ** (key.length - charIndex -
1));
  }

  return hash % arraySize;
}
```

$h(k)=2x^3+3x^2+5x+7$

$h(k)=7+x(5+x(3+x(2)))$

Horner's Rule

A polynomial

$h(k) = x_1a^{d-1} + x_2a^{d-2} + \cdots + x_{d-1}a + x_d$

can be written as a sequence of additions and multiplications

$h(k) = xd + a(x_{d-1} + a(x_{d-2} + \cdots + a(x_3 + a(x_2 + ax_1)) \cdots))$

```
function hash(key, arraySize) {
  const base = 13;

  let hash = 0;
  for (let charIndex = 0; charIndex < key.length; charIndex
+= 1) {
    const charCode = key.charCodeAt(charIndex);
    hash = (hash * base + charCode) % arraySize;
  }

  return hash;
}
```

Modular Division

- In the case that we can view the key k as a single integer, we can use modular division to compress k into the range [0, N − 1] as follows:

$$h(k) = k \bmod N$$

- If we take N to be a prime number, then the division compression map helps “spread out” the distribution of hashed values

- Take the example of keys: {200,205,210,215,220,…,600}
  - N=100 gives {0,5,10,15,20,…,0}. Repeats every 100 numbers
  - N=101 gives {98,3,8,13,18,…,95}

The Birthday Paradox

How many people must be in a room before the probability that some share a birthday becomes at least 50 percent?

- People often guess a large number. The answer is 23, hence the paradox (With 57 people, the probability is above 99%)

- Compute the Probability of No Shared Birthdays
  - The first person can have any birthday (365 choices), The second person must have a different birthday (364 choices out of 365) and so on
- the probability that all 23 people have different birthdays is (49.27%)

$$P(\text{No shared birthday}) = \frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \cdots \times \frac{(365-22)}{365}$$

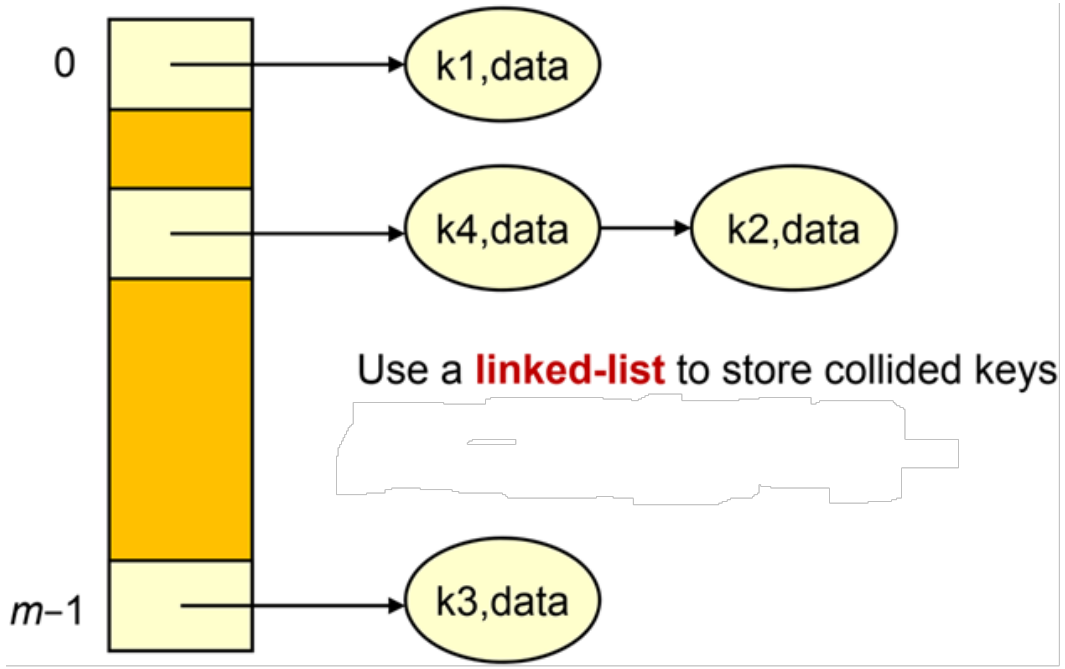
- So, the probability that at least two people share a birthday is: 50.73%

Probability of Collision

- This means that if there are 23 people in a room, the probability that some people share a birthday is 50.7%!
- In the hashing context, if we insert 23 keys into a table with 365 slots, more than half of the time we will get collisions
- Such a result is counterintuitive to many. But it shows collision is very likely

Collision Resolution: Separate Chaining

- n: number of keys in the hash table
- m: size of the hash tables — number of slots
- α: load factor α = n / m
- Measures how full the hash table is
- In separate chaining, table size equals the number of linked lists, so α is the average length of the linked lists



Hash table operations

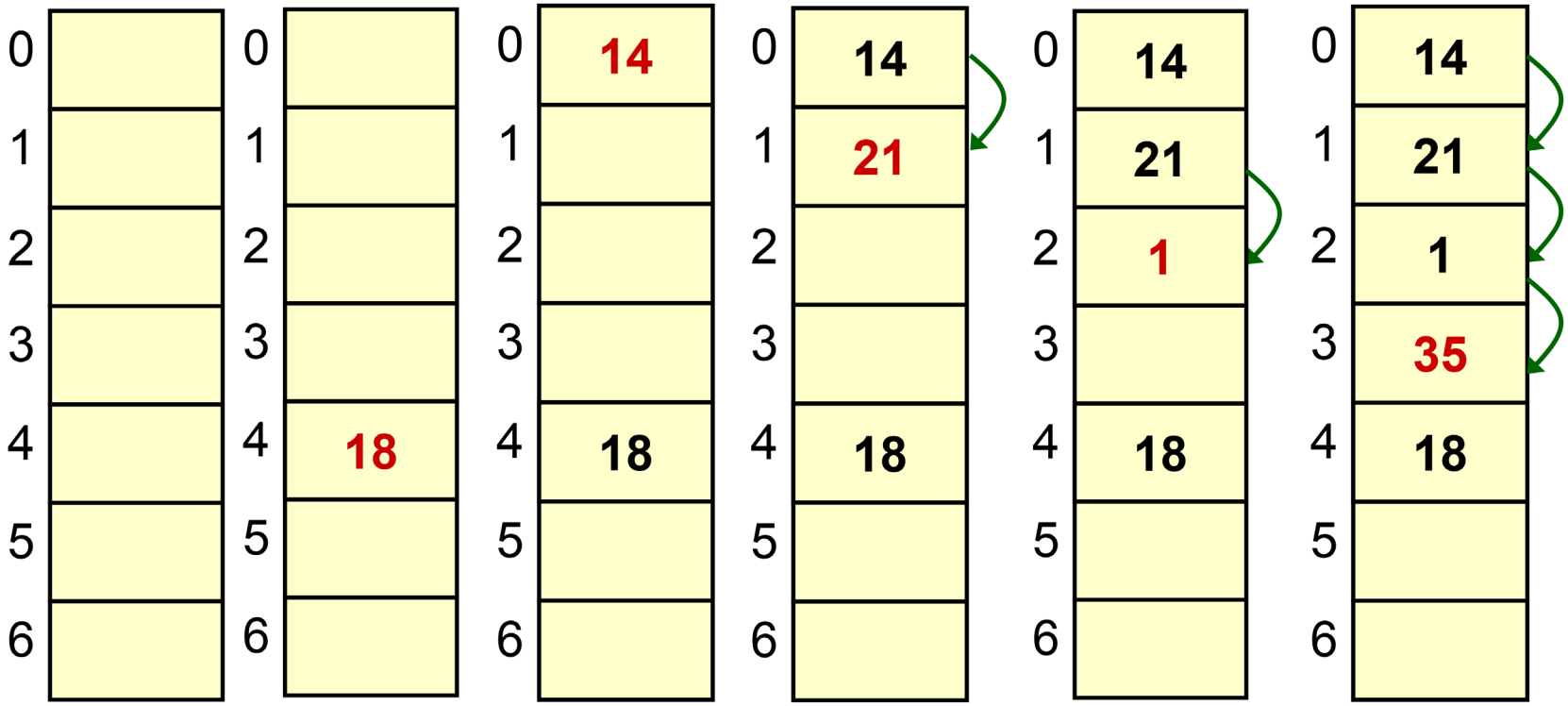
insert (key, data)
- Insert data into the list a[h(key)]
- Takes O(1) time

find (key)
- Find key from the list a[h(key)]
- Takes O(1+α) time on average

delete (key)
- Delete data from the list a[h(key)]
- Takes O(1+α) time on average

Linear Probing: Insert

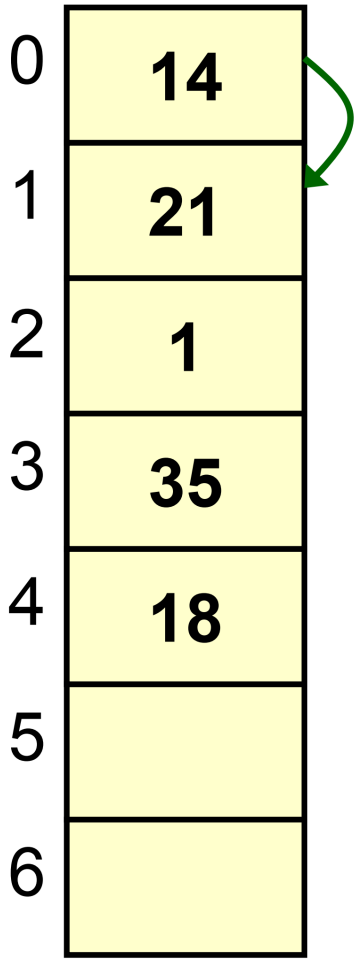
- Hashed items are in a single array. Hash code gives the home address
  - Collision is resolved by checking multiple positions
  - Each check is called a probe into the table



hash(k) = k mod 7

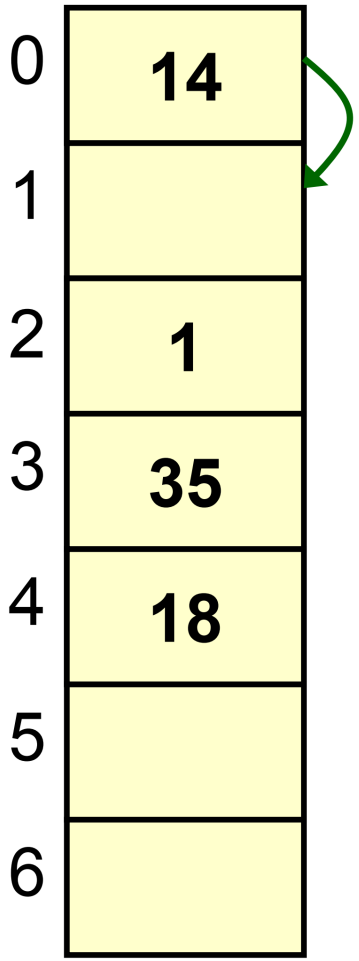
In linear probing, when there is a collision, we scan forwards for the the next empty slot (wrapping around when we reach the last slot)

Linear Probing: Delete



hash(k) = k mod 7

Delete 21:
hash(21)
= 21 mod 7
= 0



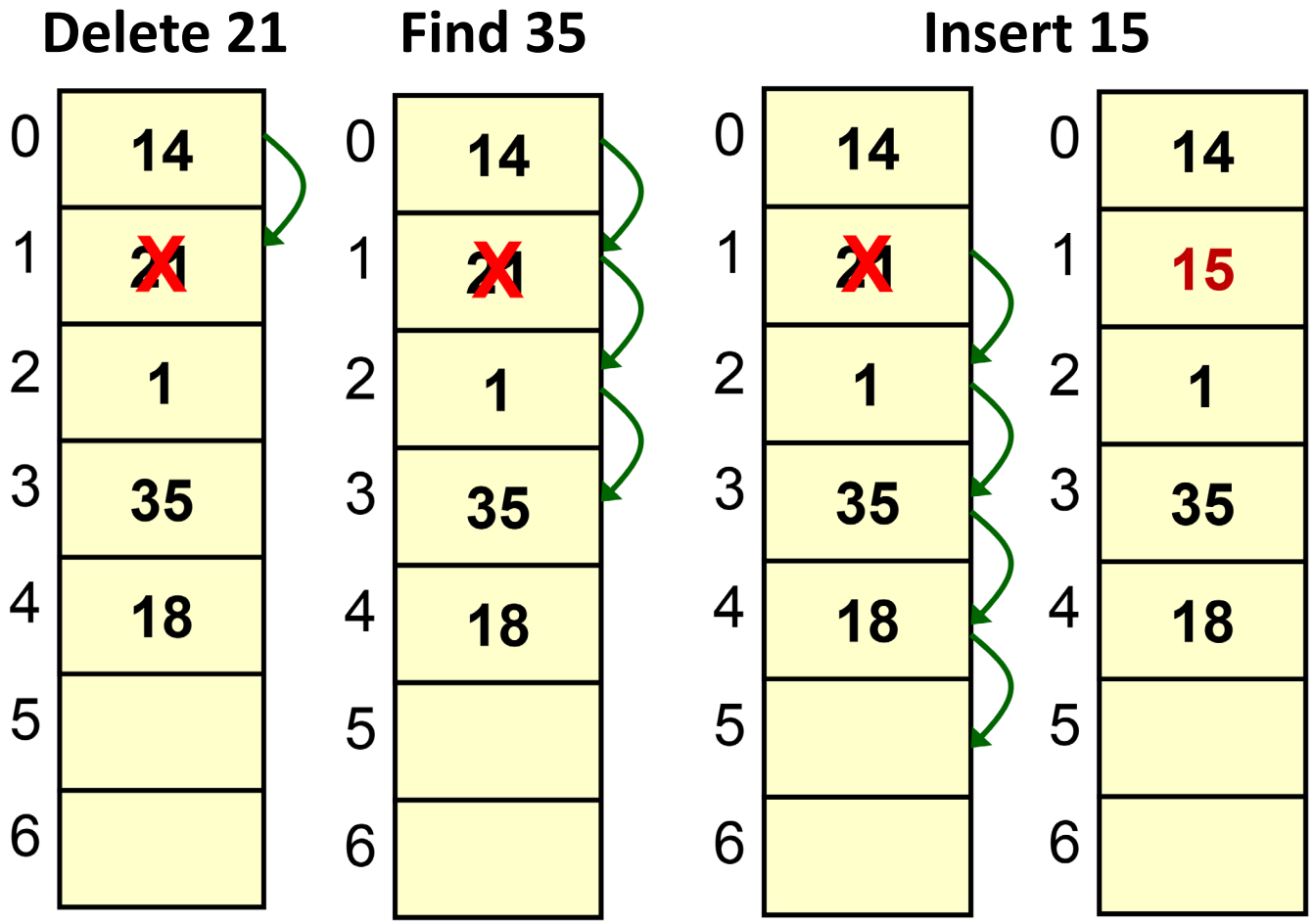
Find 35:
hash(35)
= 35 mod 7
= 0

35 Not Found!

Cannot simply remove a value

Lazy Deletion

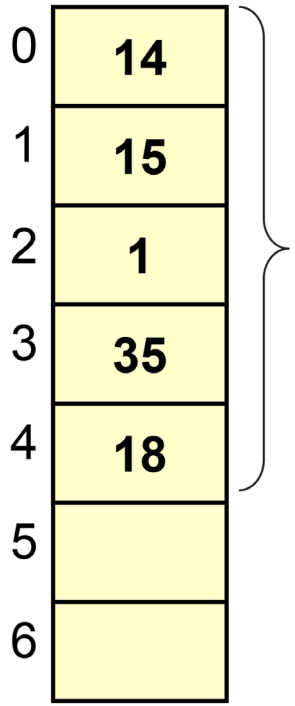
- Use three different states at each slot
  - Occupied
  - Deleted
  - Empty
- When a value is removed from linear probed hash table, we just mark the status of the slot as “deleted”, instead of emptying the slot
- Need to use a state array the same size as the hash table



We continue to search for 15, and found that 15 is not in the hash table (total 5 probes).
So, we insert this new value 15 into the slot that has been marked as deleted (i.e. slot 1)

Primary Clustering

- A cluster is a collection of consecutive occupied slots
- A cluster that covers the home address of a key is called the primary cluster of the key
- Linear probing can create large primary clusters that will increase the running time of find/insert/delete operations



consecutive occupied slots

The **probe sequence** of this linear probing is

hash(key)

(hash(key) + **1**) % *m*

(hash(key) + **2**) % *m*

(hash(key) + **3**) % *m*

⋮

- If there is an empty slot, we are sure to find it
- When an empty slot is found, conflict resolved, but the primary cluster of the key is expanded as a result
- The size of the resulting primary cluster may be very big due to the annexation of the neighboring cluster

Modified Linear Probing

To reduce primary clustering, we can modify the probe sequence to

hash(key)
(hash(key) + **1** * ***d***) % *m*
(hash(key) + **2** * ***d***) % *m*
(hash(key) + **3** * ***d***) % *m*
⋮

where ***d*** is some constant integer >1 and is co-prime to *m*
Since ***d*** and *m* are co-primes, the probe sequence covers all the slots in the hash table

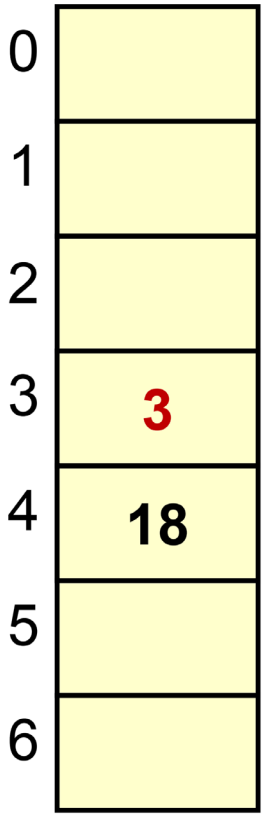
Quadratic Probing

The probe sequence of **quadratic probing** is

hash(key)
(hash(key) + **1**) % *m*
(hash(key) + **4**) % *m*
(hash(key) + **9**) % *m*
⋮
(hash(key) + i^2) % *m*

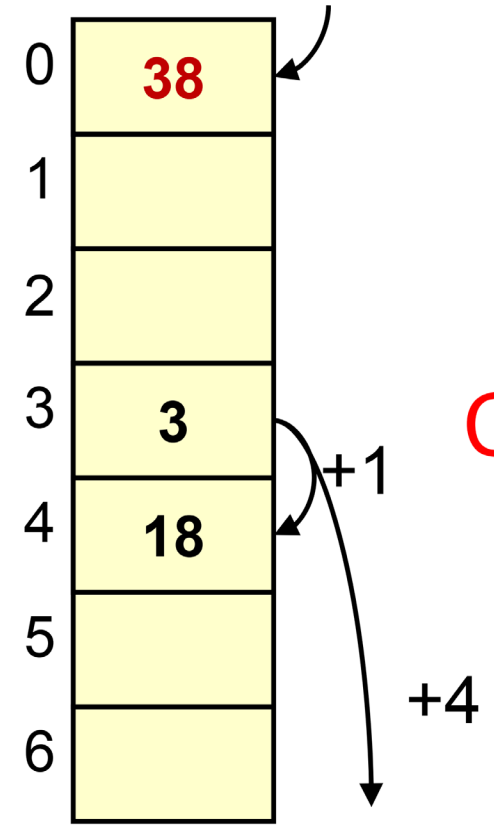
hash(*k*) = *k* mod 7

hash(18) = 4
hash(3) = 3



hash(*k*) = *k* mod 7

hash(38) = 3



Collision!

Secondary Clustering

In quadratic probing, clusters are formed along the path of probing, instead of around the home location

- These clusters are called **secondary clusters**
- Secondary clusters are formed as a result of using the same pattern in probing by all keys
  - If two keys have the same home location, their probe sequences are going to be the same

To reduce secondary clustering, we can use a second hash function to generate different probe sequences for different keys

hash(key)
(hash(key) + **1** * **$hash_2$**(key)) % *m*
(hash(key) + **2** * **$hash_2$**(key)) % *m*
(hash(key) + **3** * **$hash_2$**(key)) % *m*
⋮

$hash_2$ is called the **secondary hash function**

Double Hashing

The drawback of linear and quadratic probing is that collision resolution strategies follow the same path from a collision point regardless of key value

A better solution is double hashing:

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \% m$$

i = the number of collisions so far

h_1 = primary hash function

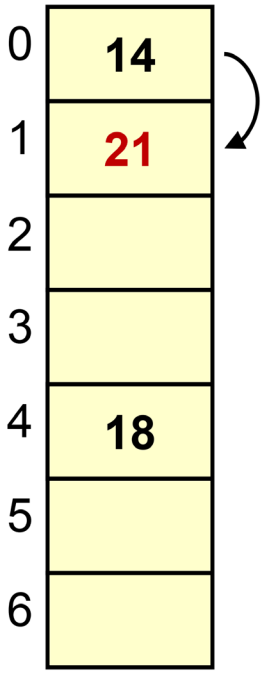
m = table size

h_2 = secondary hash function

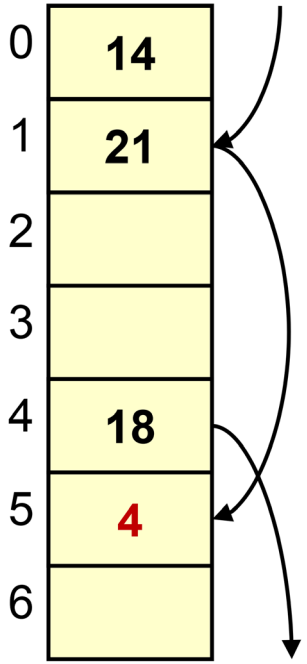
Double Hashing

hash(*k*) = *k* mod 7
$hash_2$(*k*) = *k* mod 5

hash(21) = 0
$hash_2$(21) = 1

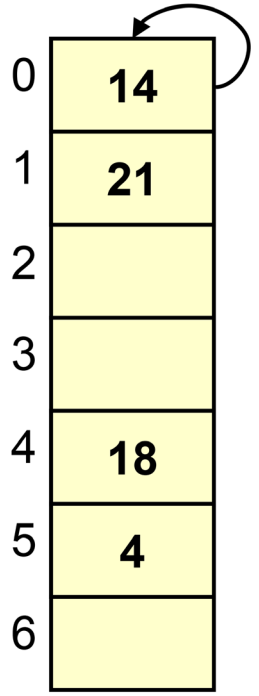


Insert 21



Insert 4

If we insert 4, the probe sequence is 4 (home), 8, 12, …



Insert 35

But if we insert 35, the probe sequence is **0, 0, 0,** …

What is wrong?
Since $hash_2$(35)=**0**.
Not acceptable!

$hash_2$(key) must not be 0

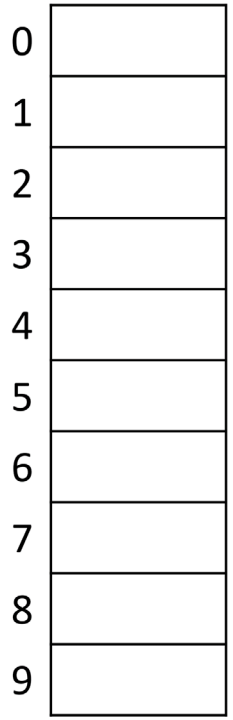
Question

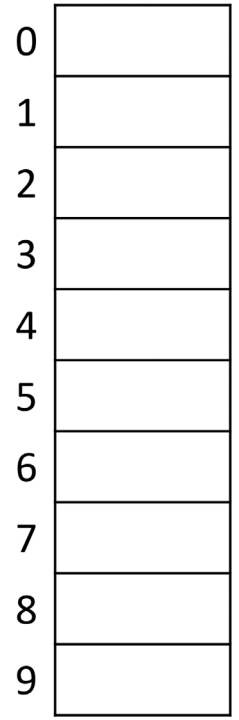
Insert (12, 13, 43, 52, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing

quadratic

$H(k, i) = (k\%m + i^2) \% m$

Total collisions = 0





double

$H(k, i) = (k\%m + i(1+k\%(m-1)) \% m$

Total collisions = 0

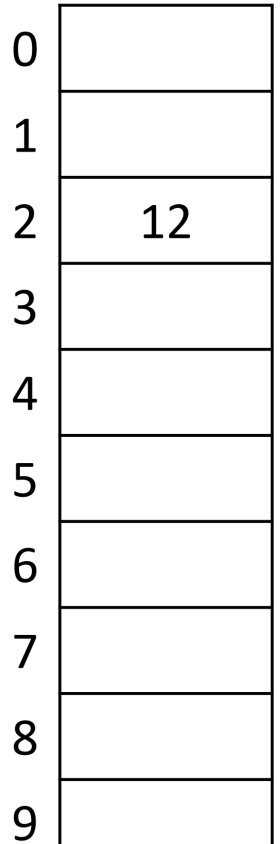
Solution

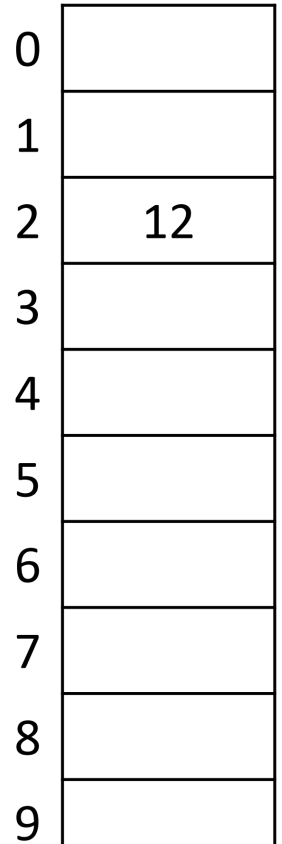
Insert (12, 13, 43, 52, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(12, 0) = (k%m + i^2) % m = 2

Total collisions = 0





double

H(12, 0) = (k%m + i(1+k%(m-1)) % m = 2

Total collisions = 0

Insert (~~12~~, 13, 43, 52, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(13, 0) = (k%m + i^2) % m = 3

Total collisions = 0

0	
1	
2	12
3	13
4	
5	
6	
7	
8	
9	

0	
1	
2	12
3	13
4	
5	
6	
7	
8	
9	

double

H(13, 0) = (k%m + i(1+k%(m-1)) % m = 3

Total collisions = 0

Insert (~~12~~, ~~13~~, 43, 52, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(43, 0) = (k%m + i^2) % m = 3

Total collisions = 1

0	
1	
2	12
3	13
4	
5	
6	
7	
8	
9	

0	
1	
2	12
3	13
4	
5	
6	
7	
8	
9	

double

H(43, 0) = (k%m + i(1+k%(m-1)) % m = 3

Total collisions = 1

Insert (~~12~~, ~~13~~, ~~43~~, 52, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(43, 1) = (k%m + i^2) % m = 4

Total collisions = 1

0	
1	
2	12
3	13
4	43
5	
6	
7	
8	
9	

0	
1	43
2	12
3	13
4	
5	
6	
7	
8	
9	

double

H(43, 1) = (k%m + i(1+k%(m-1)) % m = 1

Total collisions = 1

Insert (~~12~~, ~~13~~, ~~43~~, 52, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(52, 0) = (k%m + i^2) % m = 2

Total collisions = 2

0	
1	
2	12
3	13
4	43
5	
6	
7	
8	
9	

0	
1	43
2	12
3	13
4	
5	
6	
7	
8	
9	

double

H(52, 0) = (k%m + i(1+k%(m-1)) % m = 2

Total collisions = 2

Insert (~~12~~, ~~13~~, ~~43~~, 52, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(52, 1) = (k%m + i^2) % m = 3

Total collisions = 3

0	
1	
2	12
3	13
4	43
5	
6	
7	
8	
9	

0	52
1	43
2	12
3	13
4	
5	
6	
7	
8	
9	

double

H(52, 1) = (k%m + i(1+k%(m-1)) % m = 0

Total collisions = 2

Insert (~~12~~, ~~13~~, ~~43~~, ~~52~~, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(52, 2) = (k%m + i^2) % m = 6

Total collisions = 3

0	
1	
2	12
3	13
4	43
5	
6	52
7	
8	
9	

0	52
1	43
2	12
3	13
4	
5	
6	
7	
8	
9	

double

H(52, 1) = (k%m + i(1+k%(m-1)) % m = 0

Total collisions = 2

Insert (~~12~~, ~~13~~, ~~43~~, ~~52~~, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(72, 0) = (k%m + i^2) % m = 2

Total collisions = 4

0	
1	
2	12
3	13
4	43
5	
6	52
7	
8	
9	

0	52
1	43
2	12
3	13
4	
5	
6	
7	
8	
9	

double

H(72, 0) = (k%m + i(1+k%(m-1)) % m = 2

Total collisions = 3

Insert (~~12~~, ~~13~~, ~~43~~, ~~52~~, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(72, 1) = (k%m + i^2) % m = 3

Total collisions = 5

0	
1	
2	12
3	13
4	43
5	
6	52
7	
8	
9	

0	52
1	43
2	12
3	13
4	
5	
6	
7	
8	
9	

double

H(72, 1) = (k%m + i(1+k%(m-1)) % m = 3

Total collisions = 4

Insert (~~12~~, ~~13~~, ~~43~~, ~~52~~, 72, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(72, 2) = (k%m + i^2) % m = 6

Total collisions = 6

0	
1	
2	12
3	13
4	43
5	
6	52
7	
8	
9	

0	52
1	43
2	12
3	13
4	72
5	
6	
7	
8	
9	

double

H(72, 2) = (k%m + i(1+k%(m-1)) % m = 4

Total collisions = 4

Insert (~~12~~, ~~13~~, ~~43~~, ~~52~~, ~~72~~, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(72, 3) = (k%m + i^2) % m = 5

Total collisions = 6

0	
1	
2	12
3	13
4	43
5	72
6	52
7	
8	
9	

0	52
1	43
2	12
3	13
4	72
5	
6	
7	
8	
9	

double

H(72, 2) = (k%m + i(1+k%(m-1)) % m = 4

Total collisions = 4

Insert (~~12~~, ~~13~~, ~~43~~, ~~52~~, ~~72~~, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

$H(63, 0) = (k\%m + i^2) \% m = 3$

0	
1	
2	12
3	13
4	43
5	72
6	52
7	
8	
9	

Total collisions = 6

0	52
1	43
2	12
3	13
4	72
5	
6	
7	
8	
9	

double

$H(63, 0) = (k\%m + i(1+k\%(m-1)) \% m = 3$

Total collisions = 4

Insert (~~12~~, ~~13~~, ~~43~~, ~~52~~, ~~72~~, 63) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

H(63, 1) = (k%m + i^2) % m = 4

Total collisions = 6

0	
1	
2	12
3	13
4	43
5	72
6	52
7	
8	
9	

0	52
1	43
2	12
3	13
4	72
5	
6	
7	
8	
9	

double

H(63, 1) = (k%m + i(1+k%(m-1)) % m = 4

Total collisions = 4

Insert (~~12~~, ~~13~~, ~~43~~, ~~52~~, ~~72~~, ~~63~~) with h(k) = k%10, using first quadratic probing then double hashing.

quadratic

$H(63, 2) = (k\%m + i^2) \% m = 7$

0	
1	
2	12
3	13
4	43
5	72
6	52
7	63
8	
9	

Total collisions = 6

0	52
1	43
2	12
3	13
4	72
5	63
6	
7	
8	
9	

double

$H(63, 2) = (k\%m + i(1+k\%(m-1)) \% m = 5$

Total collisions = 4

Theorem

If quadratic probing is used, and the **table size is prime**, then a new element can always be inserted if the table is at least half empty

Proof
Let the table size, *TableSize,* be an (odd) prime greater than 3. We show that the first *TableSize/*2 alternative locations (including the initial location $h_0(x)$) are all distinct
Two of these locations are $h(x) + i^2$ (mod *TableSize*) and $h(x) + j^2$ (mod *TableSize*), where $0 \le i, j \le$ *TableSize/*2. Suppose, for the sake of contradiction, that these locations are the same, but $i \neq j$

Theorem

≡ means "congruent modulo TableSize"—a shorthand for saying the two expressions give the same remainder when divided by TableSize

Then

$h(x) + i^2 \equiv h(x) + j^2 \pmod{\text{TableSize}}$

$i^2 \equiv j^2 \pmod{\text{TableSize}}$

$i^2 - j^2 \equiv 0 \pmod{\text{TableSize}}$

$(i - j)(i + j) \equiv 0 \pmod{\text{TableSize}}$

If a prime number (here TableSize) divides a product, then it must divide at least one of the numbers in the product.
This means that TableSizeTableSize must divide either (i–j) or (i+j)

Since TableSize is prime, it follows that either (i − j) or (i + j) is equal to 0 (mod TableSize). Since i and j are distinct, the first option is not possible. Since $0 \le i, j \le \text{TableSize}/2$, the second option is also impossible

Thus, the first TableSize/2 alternative locations are distinct. If at most TableSize/2 positions are taken, then an empty spot can always be found

Universal Hashing

- If you use chaining, an adversary who knows your hash function could devise a set of keys that all hash to the same position, creating a list for the whole collection of data items (search and insert become O(n))

- **Solution**: choose the hash function randomly before creating the hash table, that is independent of keys to be stored

Universal Hashing

A universal set of hash functions H is a set of hash functions such that if you pick keys k and j at random, and choose a hash function h randomly from H, then the chance of h(k) = h(j) is no more than 1/m (where m is the size of the hash table)

So how does one get a universal set of hash functions?

A universal set of hash functions

Choose a prime number p big enough that every possible key k is < p, and choose your table size m < p

Now make $h_{a,b}(k) = ((ak + b)\ mod\ p)\ mod\ m$

- The parameters a and b may take on integer values up to p - 1, but a > 0

- a and b are chosen randomly at program start up

Example

Let p = 17 (prime), m = 6 (table size), a and b chosen randomly:

$$\begin{aligned} h_{3,4}(8) &= \big((3 \cdot 8 + 4)\%17\big)\%6 \\ &= (28\%17)\%6 \\ &= 11\%6 \\ &= 5 \end{aligned}$$

Choosing a and b at random as the program starts denies an adversary the ability to choose input that generates the worst case

Hash of hashes

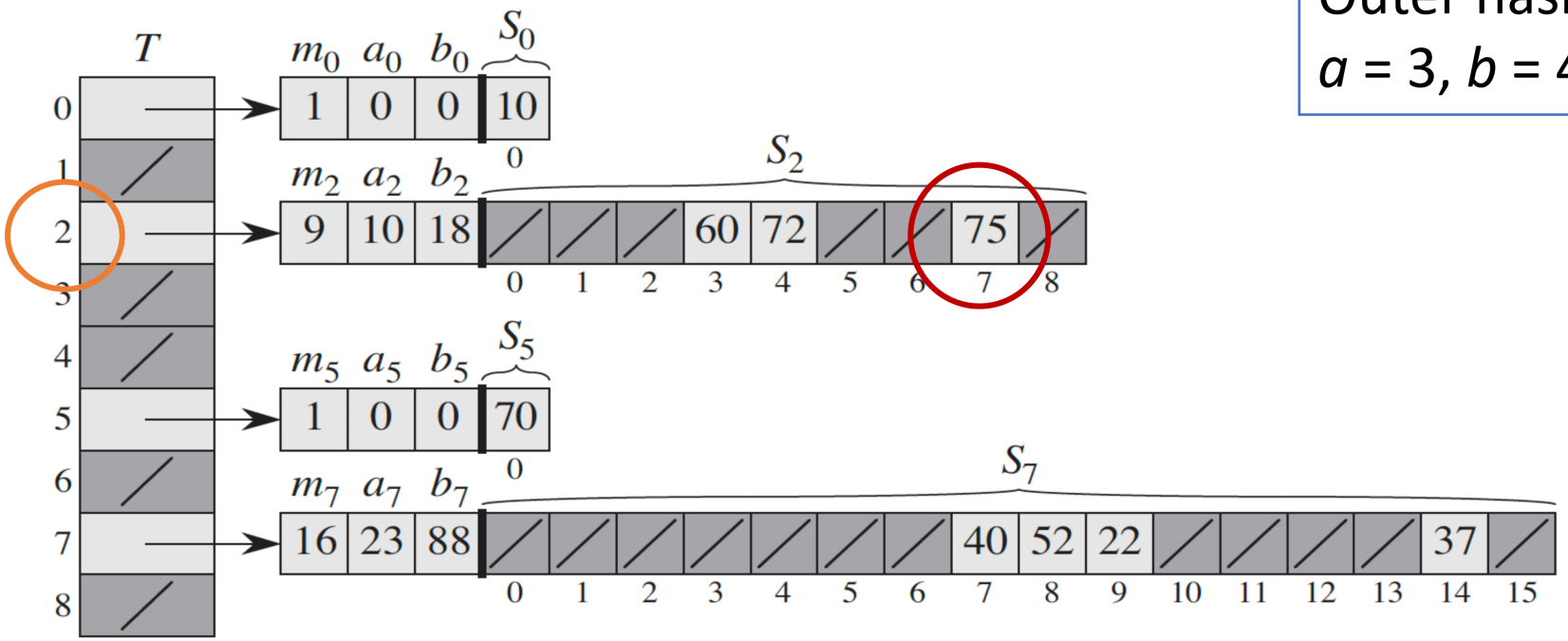
To create a perfect hashing scheme, we use two levels of hashing, with universal hashing at each level.

Instead of making a linked list of the keys hashing to slot j, however, we use a small **secondary hash table** S_j with an associated hash function h_j.

By trying several hash functions from a universal class, we can easily achieve the goal of having no collisions in the secondary tables.

Hash of hashes

K = {10, 22, 37, 40, 52, 60, 70, 72, 75}
Outer hash h(k) = (($ak + b$) % p) % m
a = 3, b = 42, p = 101, and m = 9



Example: $h(75) = 2$, and so key 75 hashes to slot 2 of table T. A secondary hash table S_j stores all keys hashing to slot j. The size of hash table S_j is $m_j = n_j^2$, and the associated hash function is $h_j(k) = \left((a_j k + b_j)\%p\right)\%m_j$

Since $h_2(75) = 7$, key 75 is stored in slot 7 of secondary hash table S_2.

Perfect Hashing

- Perfect hashing: No Collisions
  - Only possible when we know the keys in advance i.e. static keys
  - Example: keywords in a programming language, words in a dictionary, Network routing tables
- If the set of keys is static, we can design the hash function to get a worst-case search = O(1). This technique is called perfect hashing
- Two-Level Perfect Hashing
  - First Level: Hash function maps keys to buckets
  - Second Level: Each bucket gets its own hash function with no collisions

Perfect Hashing

Main hash table: choose size $m = n$ where n is the number of data items, choose prime $p >$ the biggest key value. Choose a and b randomly to get primary hash function h(k) = ((ak + b)%p)%m.

Test h as follows:

1. For each key k, work out the home cell $h(k)$. Keep a count n_i for each cell i of how many keys hash to i.
2. Check whether space required is too big: is the sum of all the n_i^2 giving a total > $2n$? Then h is not good enough so repeat the process with new a, b for a new primary hash function.

Perfect hashing

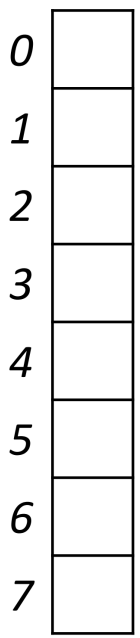
If the primary hash function *h* is good enough:

1. For each slot *i*, get the secondary hash function h_i by setting $p_i = p$ (i.e. p doesn't change), setting $m_i = n_i^2$, and choosing a_i and b_i randomly.
2. Check to make sure that the resulting h_i doesn't cause any collisions within the secondary table.

Perfect hashing example

K = {8, 22, 36, 75, 61, 13, 84, 58}

$$h(k) = \big((ak + b)\%p\big)\%\, m$$
$$h_j(k) = \Big(\big(a_j k + b_j\big)\%p\Big)\,\%m$$



Step 1: Randomly generate values for *a* and *b*, select $p > K$

Step 2: For each key *k*, work out home cell *h*(*k*). Keep a count.

Step 3: Check if $\sum_{j=0}^{m-1} n_j^2 < 2n$

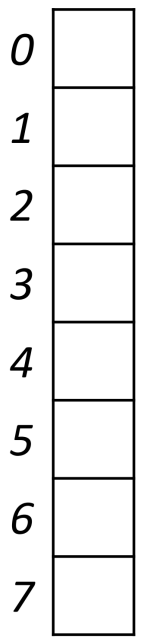
Step 4: Populate sub-tables, calculating new hash functions

Perfect hashing example

K = {8, 22, 36, 75, 61, 13, 84, 58}

p = 87, a = 64, b = 5

$$h(k) = \big((ak + b)\%p\big)\%\, m$$
$$h_j(k) = \Big(\big(a_j k + b_j\big)\%p\Big)\,\%m$$



Step 1: Randomly generate values for a and b, select $p > K$

Step 2: For each key k, work out home cell $h(k)$. Keep a count.

Step 3: Check if $\sum_{j=0}^{m-1} n_j^2 < 2n$

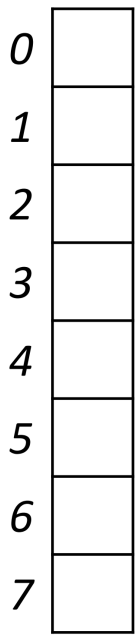
Step 4: Populate sub-tables, calculating new hash functions

Perfect hashing example

K = {8, 22, 36, 75, 61, 13, 84, 58}

p = 87, a = 64, b = 5

$$h(k) = \big((ak + b)\%p\big)\%\, m$$
$$h_j(k) = \Big(\big(a_j k + b_j\big)\%p\Big)\,\%m$$



Step 1: Randomly generate values for a and b, select $p > K$

Step 2: For each key k, work out home cell $h(k)$. Keep a count.

Step 3: Check if $\sum_{j=0}^{m-1} n_j^2 < 2n$

Step 4: Populate sub-tables, calculating new hash functions

Perfect hashing example

K = {8, 22, 36, 75, 61, 13, 84, 58}

p = 87, a = 64, b = 5

$$h(k) = \big((ak + b)\%p\big)\%\, m$$
$$h_j(k) = \Big(\big(a_j k + b_j\big)\%p\Big)\,\%m$$

Step 1: Randomly generate values for a and b, select $p > K$

Step 2: For each key k, work out home cell $h(k)$. Keep a count.

Step 3: Check if $\sum_{j=0}^{m-1} n_j^2 < 2n$

Step 4: Populate sub-tables, calculating new hash functions

0	/	0
1	/	61
2	/	8, 84
3	/	0
4	/	75
5	/	22
6	/	13
7	/	36, 58

Perfect hashing example

K = {8, 22, 36, 75, 61, 13, 84, 58}

p = 87, a = 64, b = 5

$$h(k) = \big((ak + b)\%p\big)\%\, m$$
$$h_j(k) = \Big(\big(a_j k + b_j\big)\%p\Big)\,\%m$$

			Count
0	/	0	0
1	/	61	1
2	/	8, 84	2
3	/	0	0
4	/	75	1
5	/	22	1
6	/	13	1
7	/	36, 58	2

Step 1: Randomly generate values for a and b, select $p > K$

Step 2: For each key k, work out home cell $h(k)$. Keep a count.

Step 3: Check if $\sum_{j=0}^{m-1} n_j^2 < 2n$

Step 4: Populate sub-tables, calculating new hash functions

Perfect hashing example

K = {8, 22, 36, 75, 61, 13, 84, 58}

$p = 87$, $a = 64$, $b = 5$

$$h(k) = \big((ak + b)\%p\big)\%\, m$$
$$h_j(k) = \Big(\big(a_j k + b_j\big)\%p\Big)\%m$$

			Count
0	/	0	0
1	/	61	1
2	/	8, 84	2
3	/	0	0
4	/	75	1
5	/	22	1
6	/	13	1
7	/	36, 58	2

Step 1: Randomly generate values for *a* and *b*, select $p > K$

Step 2: For each key *k*, work out home cell $h(k)$. Keep a count.

Step 3: Check if $\sum_{j=0}^{m-1} n_j^2 < 2n$

Step 4: Populate sub-tables, calculating new hash functions

$$\sum_{j=0}^{m-1} n_j^2 = 0 + 1 + 4 + 0 + 1 + 1 + 1 + 4 = 12$$

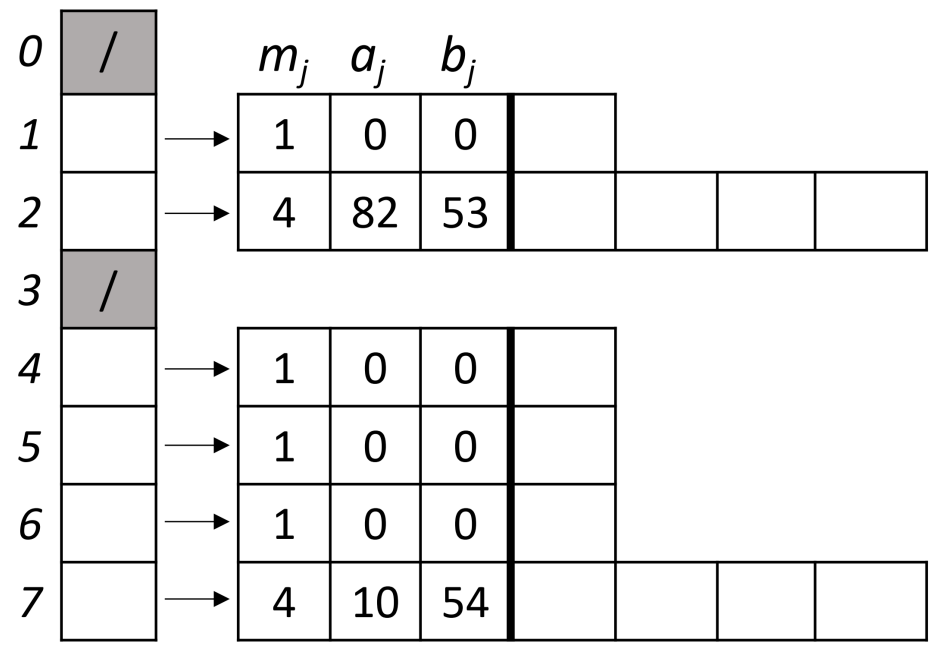
$$12 < 2 * n = 16$$ ✓

Perfect hashing example

K = {8, 22, 36, 75, 61, 13, 84, 58}

p = 87, a = 64, b = 5

$$h(k) = \big((ak + b)\%p\big)\%\, m$$
$$h_j(k) = \Big(\big(a_j k + b_j\big)\%p\Big)\,\%m$$



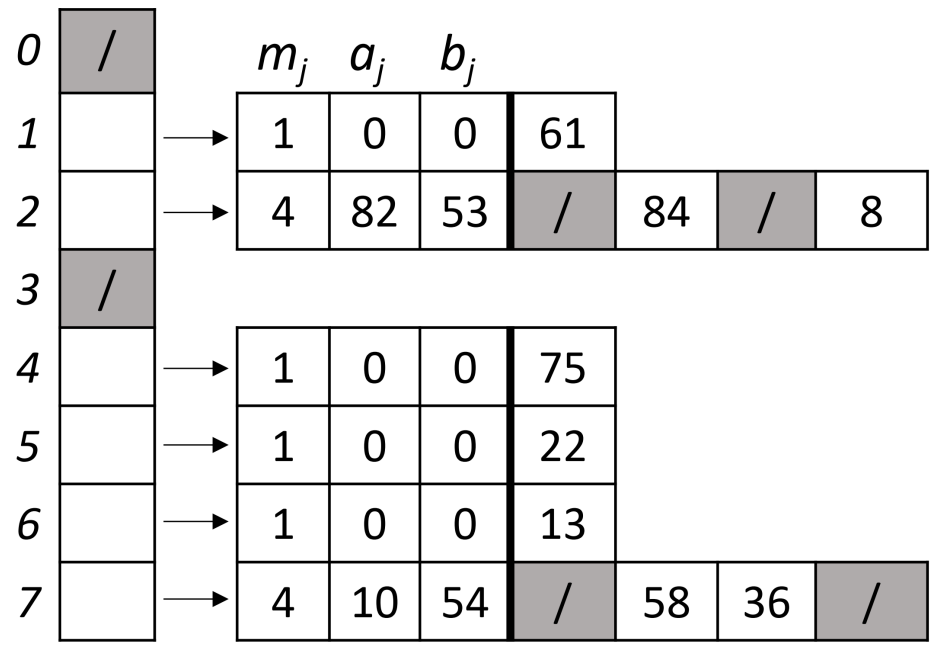
Step 4: Populate sub-tables, calculating new hash functions

Perfect hashing example

K = {8, 22, 36, 75, 61, 13, 84, 58}

$p = 87$, $a = 64$, $b = 5$

$$h(k) = \big((ak + b)\%p\big)\%\, m$$
$$h_j(k) = \Big(\big(a_j k + b_j\big)\%p\Big)\,\%m$$



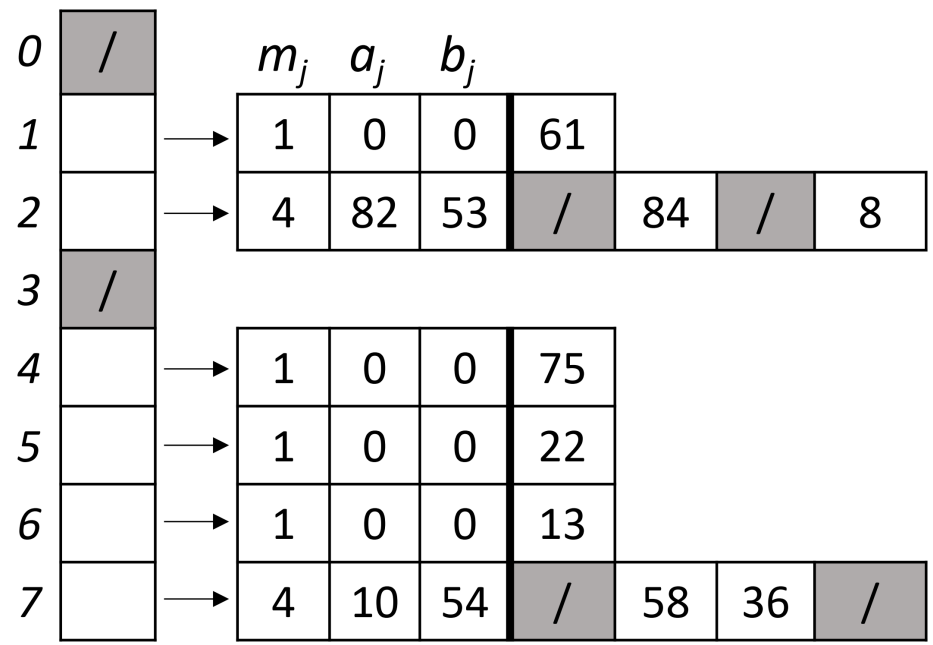
Step 4: Populate sub-tables, calculating new hash functions

Perfect hashing example

K = {8, 22, 36, 75, 61, 13, 84, 58}

$p = 87$, $a = 64$, $b = 5$

$$h(k) = \big((ak + b)\%p\big)\%\, m$$
$$h_j(k) = \Big(\big(a_j k + b_j\big)\%p\Big)\,\%m$$



No S_j keys hash to the same sub-slots. **All done!**

Cuckoo Hashing

In the balls and bins problem, if N items are randomly tossed into N bins, the size of the largest bin is expected to be Θ(log N/ log log N)

In the mid 1990s, it was shown that if, at each toss, two bins were randomly chosen and the item was tossed into the more empty bin (at the time), then the size of the largest bin would only be Θ(log log N), a significantly lower number.

Cuckoo Hashing

Suppose we have N items. We maintain two tables, each more than half empty, and we have two independent hash functions that can assign each item to a position in each table. Cuckoo hashing maintains the invariant that an item is always stored in one of these two locations

Table 1	
0	B
1	C
2	
3	E
4	

Table 2	
0	D
1	
2	A
3	
4	F

A: 0, 2

B: 0, 0

C: 1, 4

D: 1, 0

E: 3, 2

F: 3, 4

Cuckoo Hashing - Example

To insert a new item, x, first make sure it is not already there. We can then use the first hash function, and if the (first) table location is empty, the item can be placed

Table 1	
0	A
1	
2	
3	
4	

Table 2	
0	
1	
2	
3	
4	

A: 0, 2

Suppose now we want to insert B, which has hash locations 0 in Table 1 and 0 in Table 2.

Table 1	
0	B
1	
2	
3	
4	

Table 2	
0	
1	
2	A
3	
4	

A: 0, 2
B: 0, 0

C is simple. Let's look at D. with hash locations (1, 0). But the Table 1 location (position 1) is already taken. Note also that the Table 2 location is not already taken, but we don't look there. Instead, we have D replace C, and then C goes into Table 2 at position 4, as suggested by its second hash function. E is also easy.

Table 1	
0	B
1	D
2	
3	E
4	

Table 2	
0	
1	
2	A
3	
4	C

A: 0, 2
B: 0, 0
C: 1, 4
D: 1, 0
E: 3, 2

F involves many displacements

Table 1	
0	**A**
1	D
2	
3	**F**
4	

Table 2	
0	**B**
1	
2	**E**
3	
4	C

A: 0, 2
B: 0, 0
C: 1, 4
D: 1, 0
E: 3, 2
F: 3, 4

Example (contd.)

Table 1	
0	B
1	C
2	
3	E
4	

Table 2	
0	D
1	
2	A
3	
4	F

A: 0, 2
B: 0, 0
C: 1, 4
D: 1, 0
E: 3, 2
F: 3, 4

- What if we try to insert a seventh item G (1,2)?
- There are only three available locations in the first table for the six items, and there are only three available locations in the second table for the six items
- Clearly, if there were a seventh item, G, with locations 1 for Table 1 and 2 for Table 2, it could not be inserted into the table by any algorithm (the seven items would be competing for six table locations)
  - The answer in this situation would be to pick another hash function, and this can be fine as long as it is unlikely that this situation occurs

Example (contd.)

- If the table's load factor is below 0.5, an analysis shows that the probability of a cycle is very low, that the expected number of displacements is a small constant, and that it is extremely unlikely that a successful insertion would require more than O(log N) displacements
  - As such, we can simply rebuild the tables with new hash functions after a certain number of displacements are detected

Distributed Lookup

- Look up (*key, value*)
- Cooperating set of nodes store data
- Ideally:
  - No central coordinator
  - Some nodes can be down

Approaches

1. Central coordinator
   – Napster

2. Flooding
   – Gnutella

3. Distributed hash tables
   – CAN, Chord, Amazon Dynamo, Tapestry, …

1. Central Coordinator

- Example: Napster
  - Central directory
  - Identifies content (names) and the servers that host it
  - *lookup(name)* → {list of servers}
  - Download from any of available servers
    - Pick the best one by pinging and comparing response times

- Another example: GFS
  - Controlled environment compared to Napster
  - Content for a given key is broken into chunks
  - Master handles all queries … but not the data

1. Central Coordinator - Napster

- Pros
  - Super simple
  - Search is handled by a single server (master)
  - The directory server is a single point of control
    - Provides definitive answers to a query

- Cons
  - Master has to maintain state of all peers
  - Server gets all the queries
  - The directory server is a single point of control
    - No directory, no service!

2. Query Flooding

- Example: Gnutella distributed file sharing

- Well-known nodes act as anchors
  - Nodes with files inform an anchor about their existence
  - Nodes select other nodes as peers

2. Query Flooding

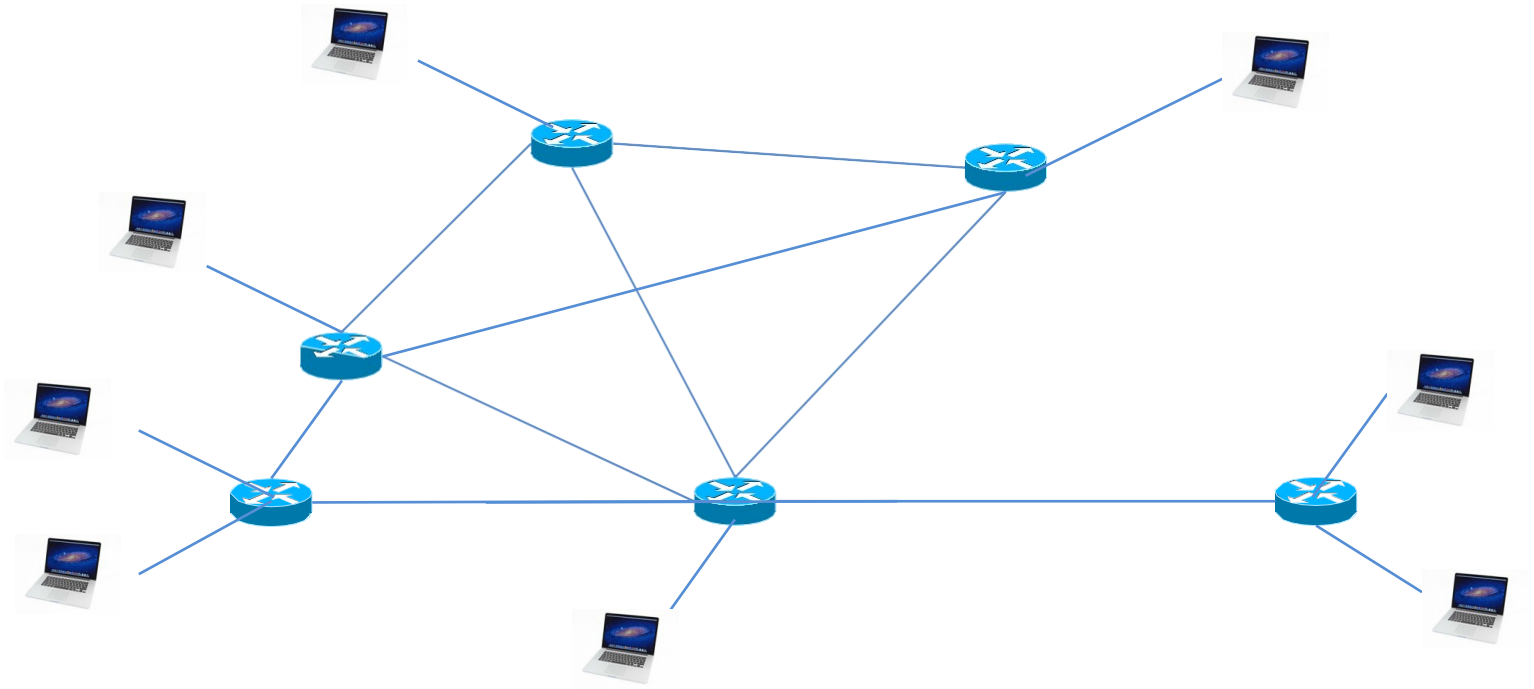
- Send a query to peers if a file is not present locally
  - Each request contains:
    - Query key
    - Unique request ID
    - Time to Live (TTL, maximum hop count)

- Peer either responds or routes the query to its neighbors
  - Repeat until TTL = 0 or if the request ID has been processed
  - If found, send response (node address) to the requestor
  - **Back propagation**: response hops back to reach originator

Overlay network

An overlay network is a virtual network formed by peer connections

- Any node might know about a small set of machines
- “Neighbors” may not be physically close to you

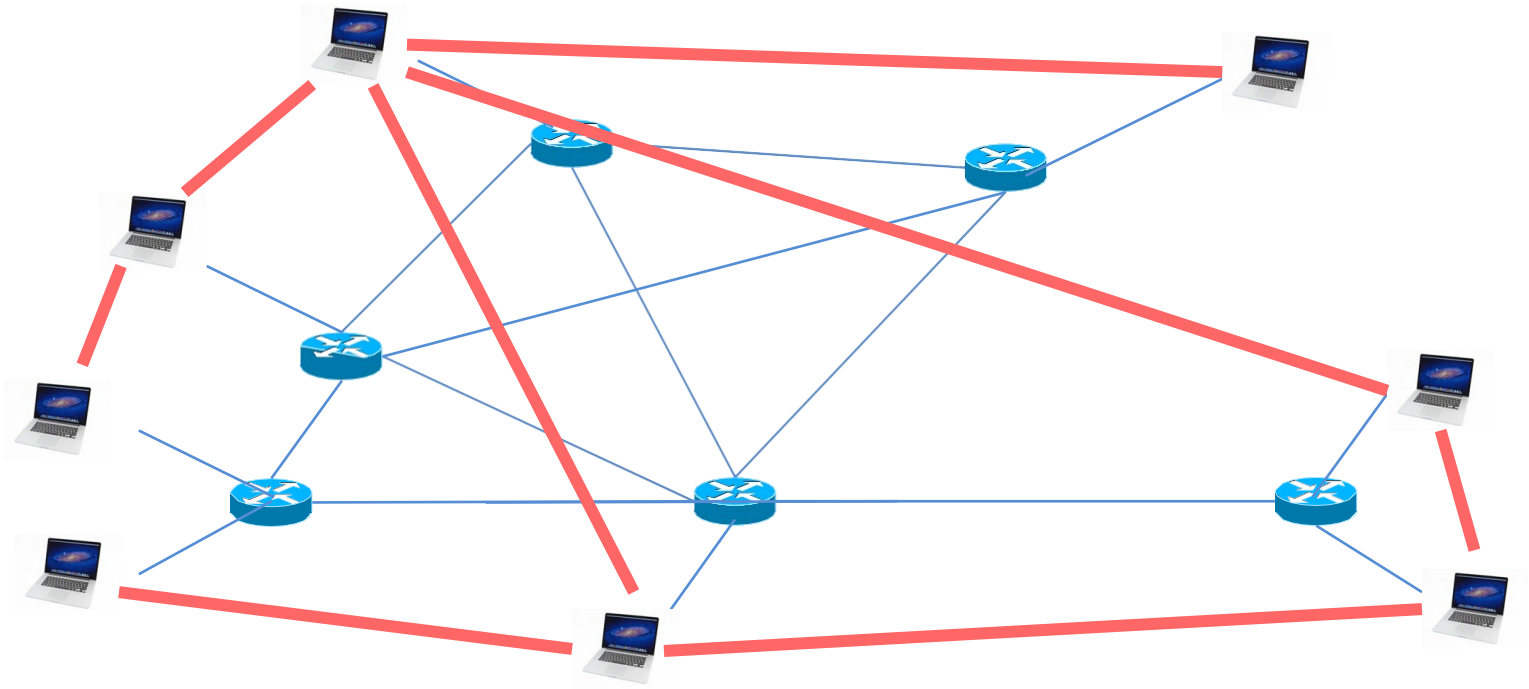


Underlying IP Network

Overlay network

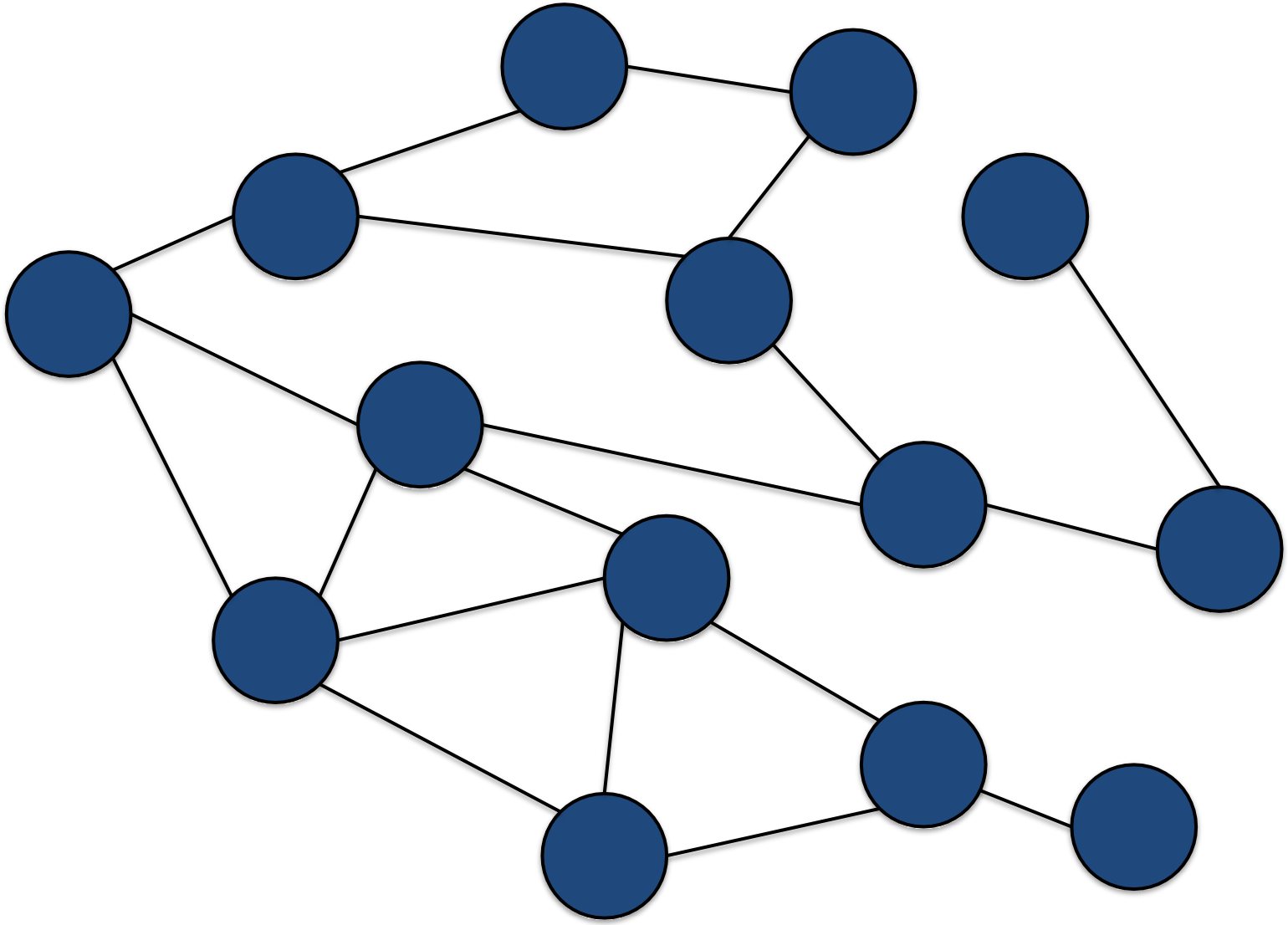
An overlay network is a virtual network formed by peer connections

- Any node might know about a small set of machines
- “Neighbors” may not be physically close to you

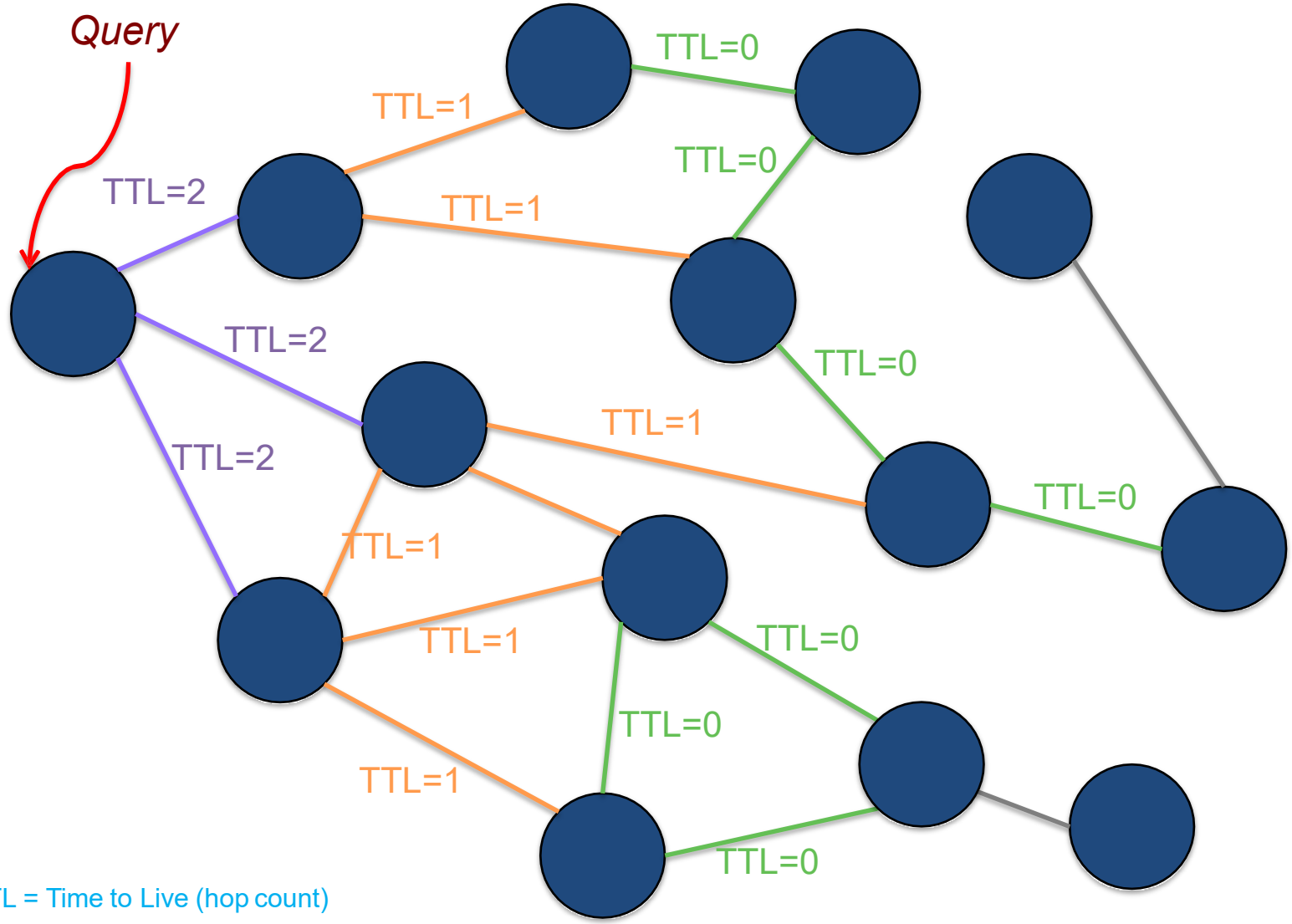


Overlay Network

Flooding Example: Overlay Network

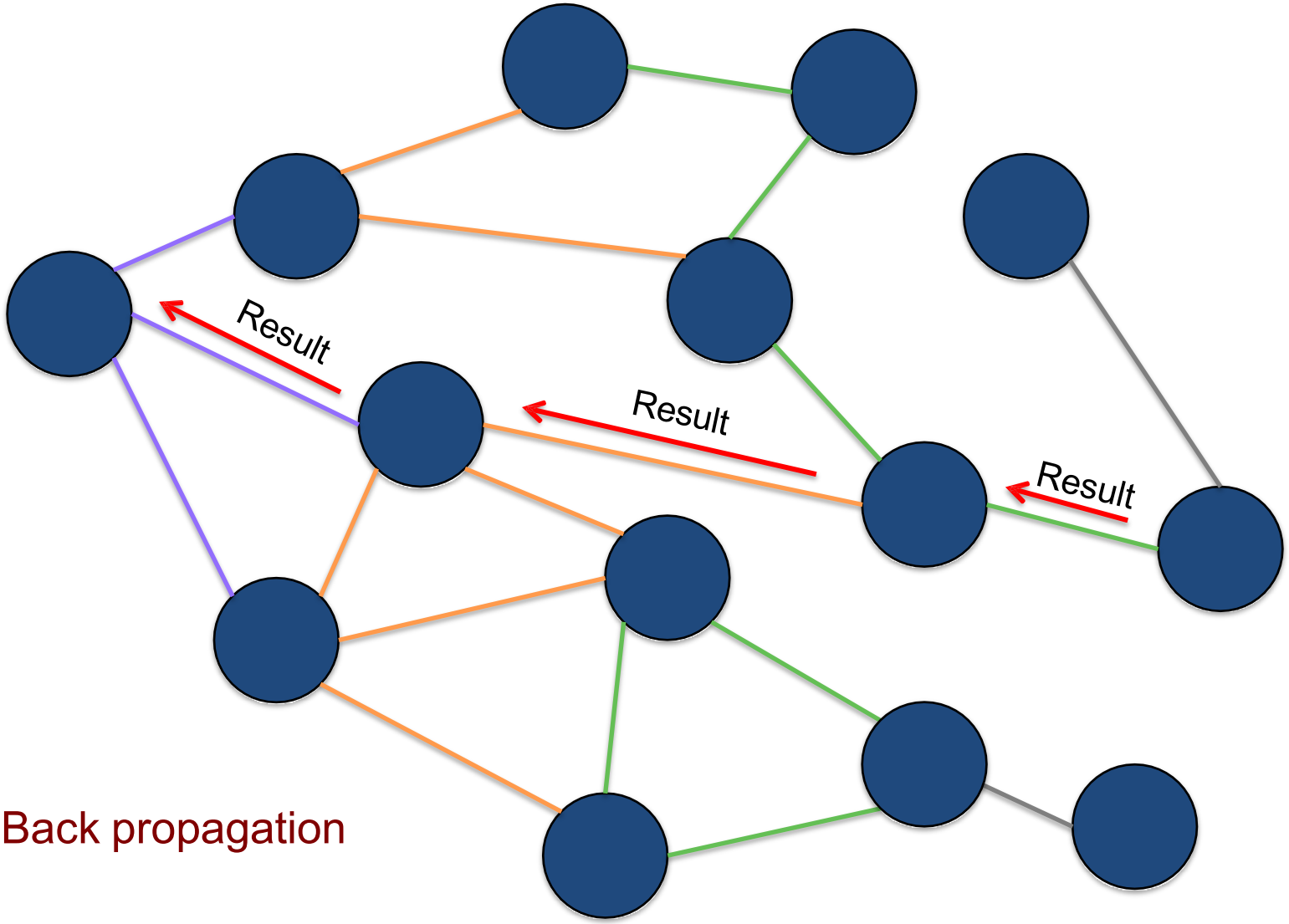


Flooding Example: Query Flood



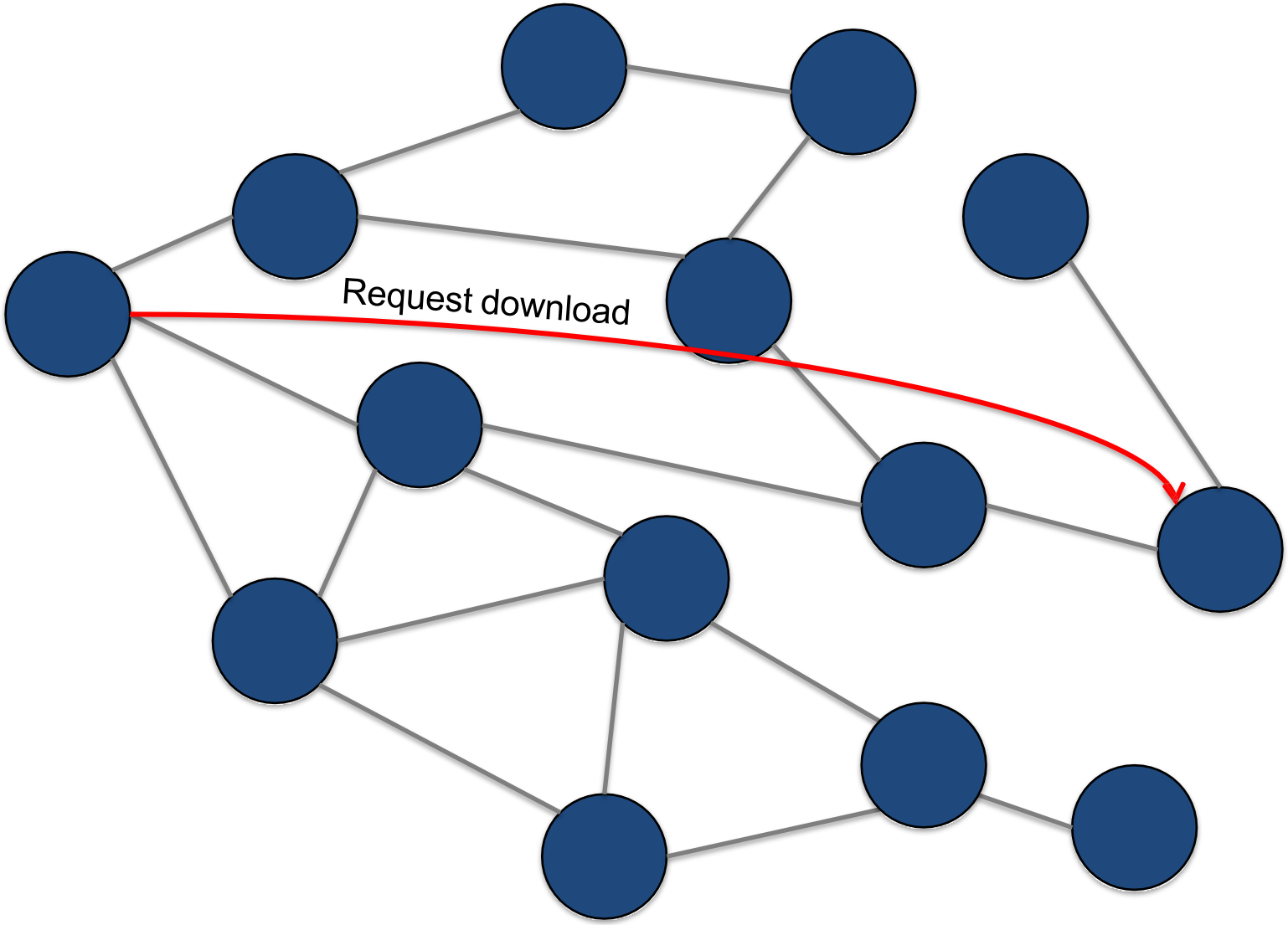
TTL = Time to Live (hop count)

Flooding Example: Query response



Back propagation

Flooding Example: Download



What's wrong with flooding?

- Some nodes are not always up and some are slower than others
  - Gnutella dealt with this by classifying some nodes as special ("ultrapeers" in Gnutella)
- Poor use of network resources
- Potentially high latency
  - Requests get forwarded from one machine to another
  - Back propagation (e.g., in Gnutella's design): replies go through the same sequence of systems used in the query, increasing latency even more – useful in preserving anonymity

3. Distributed Hash Tables

Distributed Hash Tables (DHT)

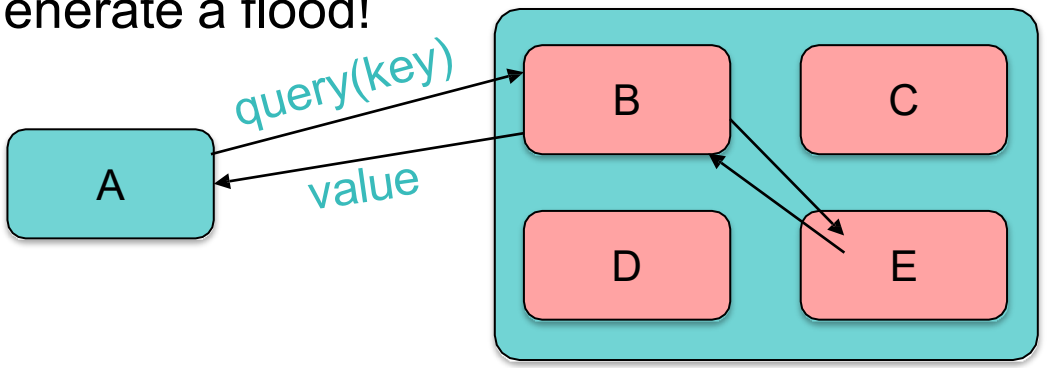
Create a peer-to-peer version of a (*key, value*) data store

How we want it to work

1. A peer (*A*) queries the data store with a key
2. The data store finds the peer (*B*) that has the value
3. That peer (*B*) returns the (key, value) pair to the querying peer (*A*)

Make it efficient!

– A query should not generate a flood!



Consistent hashing

- Conventional hashing
  - Practically all keys must be remapped if the table size changes
- Consistent hashing
  - Most keys will hash to the same value as before
  - On average, K/n keys will need to be remapped

    K = # keys, n = # of buckets

3. Distributed hashing

- Spread the hash table across multiple nodes
- Each node stores a portion of the key space

  lookup(*key*) → *node ID* that holds (*key*, *value*)

  lookup(node_ID, key) → *value*

Questions

How do we partition the data & do the lookup?

& keep the system decentralized?

& make the system scalable (lots of nodes with dynamic changes)?

& fault tolerant (replicated data)?

Distributed Hashing
Case Study

CAN: Content Addressable Network

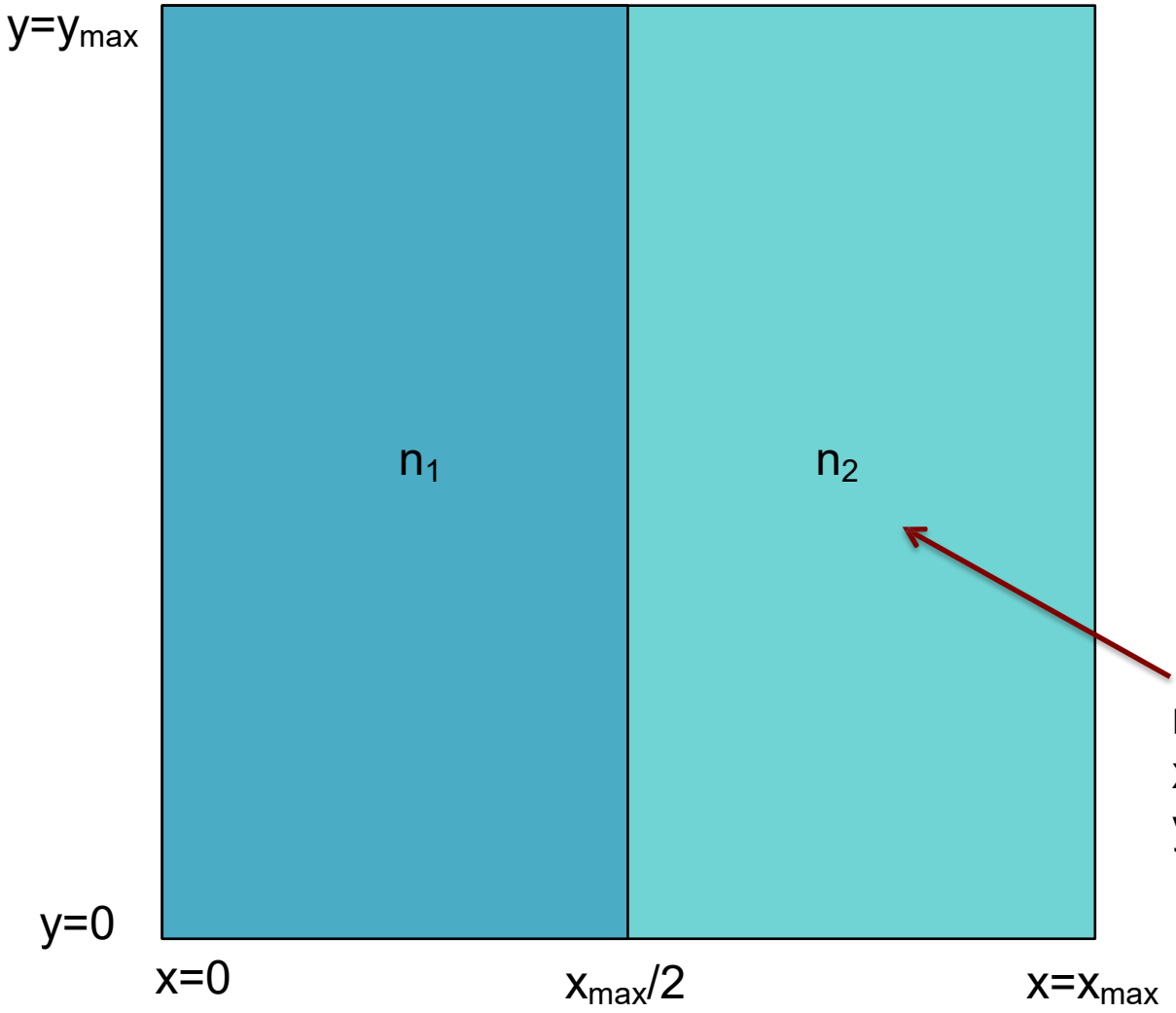
CAN design

- Create a logical grid
  - x-y in 2-D (but not limited to two dimensions)

- Separate hash function per dimension
  - h_x(key), h_y(key)

- A node
  - Is responsible for a range of values in each dimension
  - Knows its neighboring nodes

CAN *key→node* mapping: 2 nodes



$x = hash_x(key)$

$y = hash_y(key)$

if $x < (x_{max}/2)$

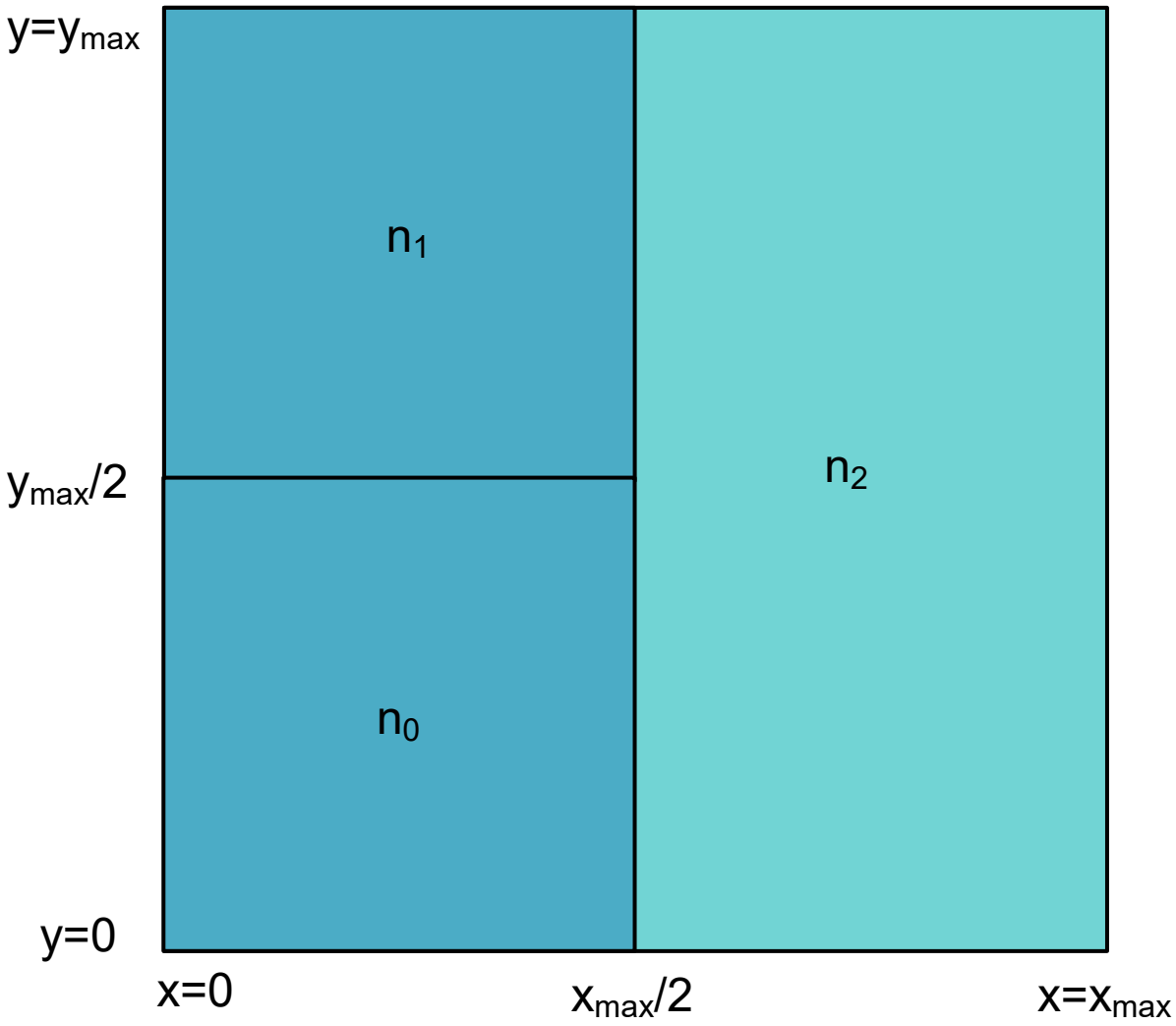
n_1 has (*key, value*)

if $x \geq (x_{max}/2)$

n_2 has (*key, value*)

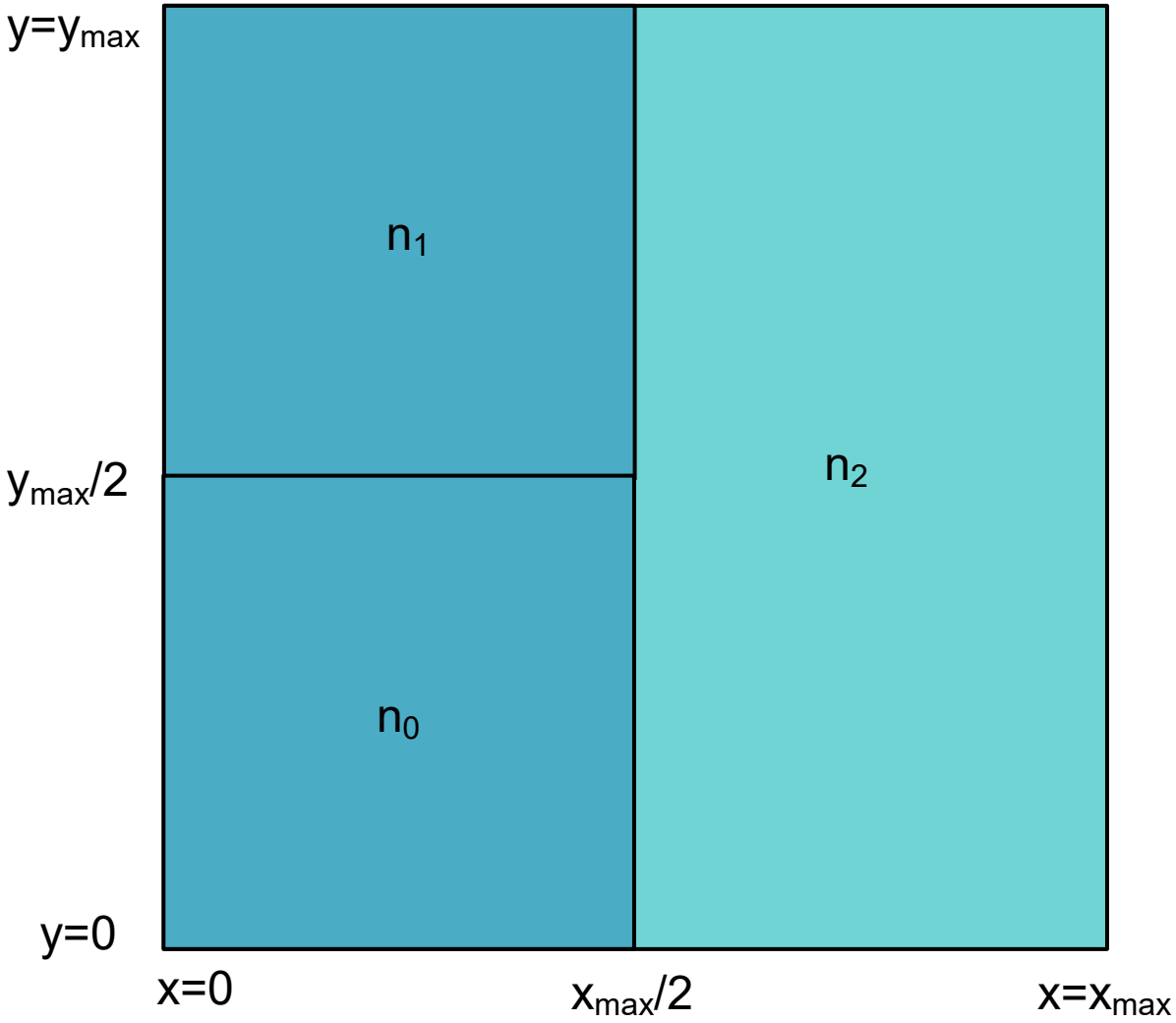
n_2 is responsible for a zone
$x=(x_{max}/2 .. x_{max})$,
$y=(0 .. y_{max})$

CAN partitioning



Any node can be split in two – either horizontally or vertically

CAN key→node mapping



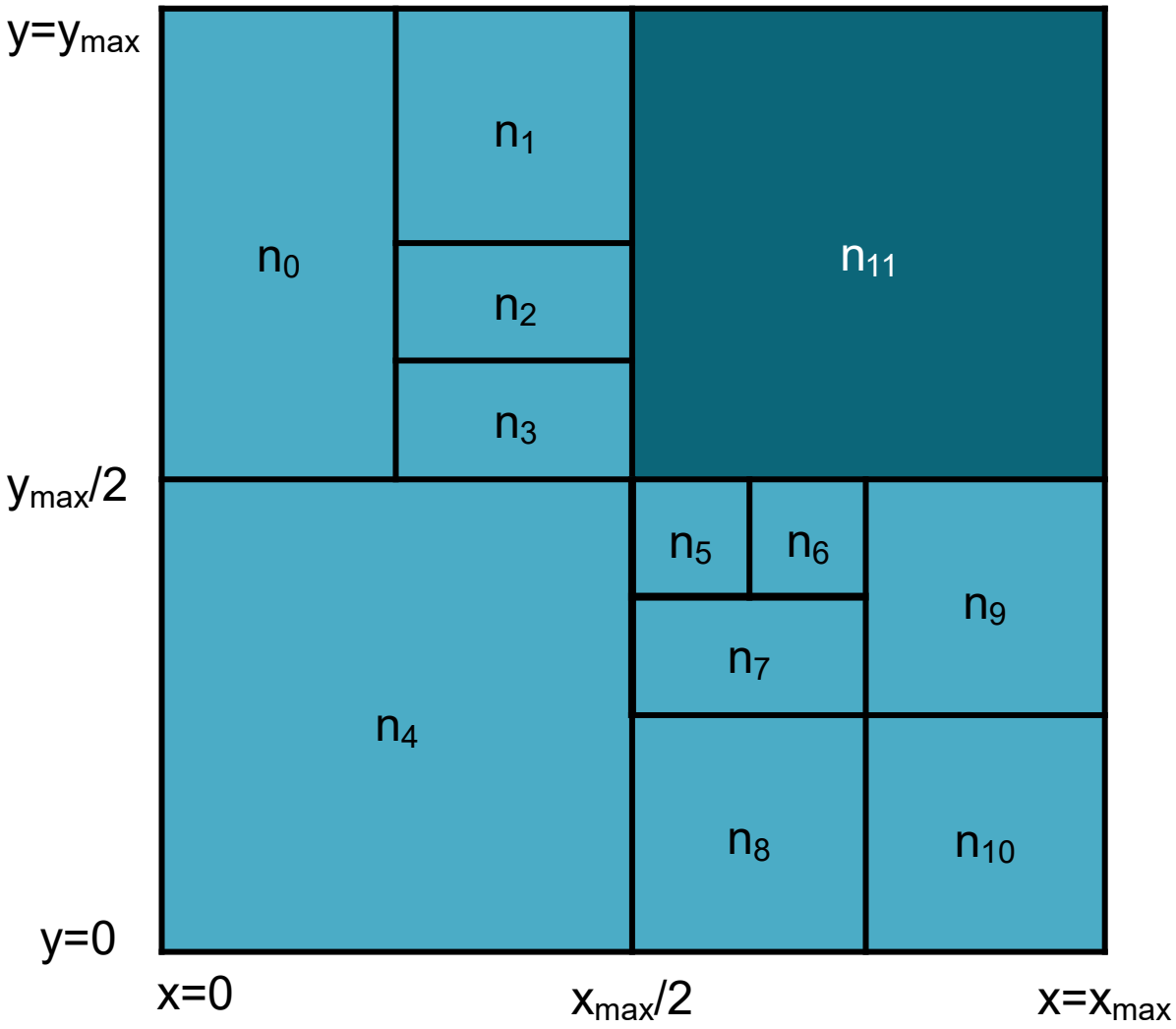
$x = hash_x(key)$

$y = hash_y(key)$

if $x < (x_{max}/2)$ {
  if $y < (y_{max}/2)$
    n_0 has (*key, value*)
  else
    n_1 has (*key, value*)
}

if $x \geq (x_{max}/2)$
    n_2 has (*key, value*)

CAN partitioning



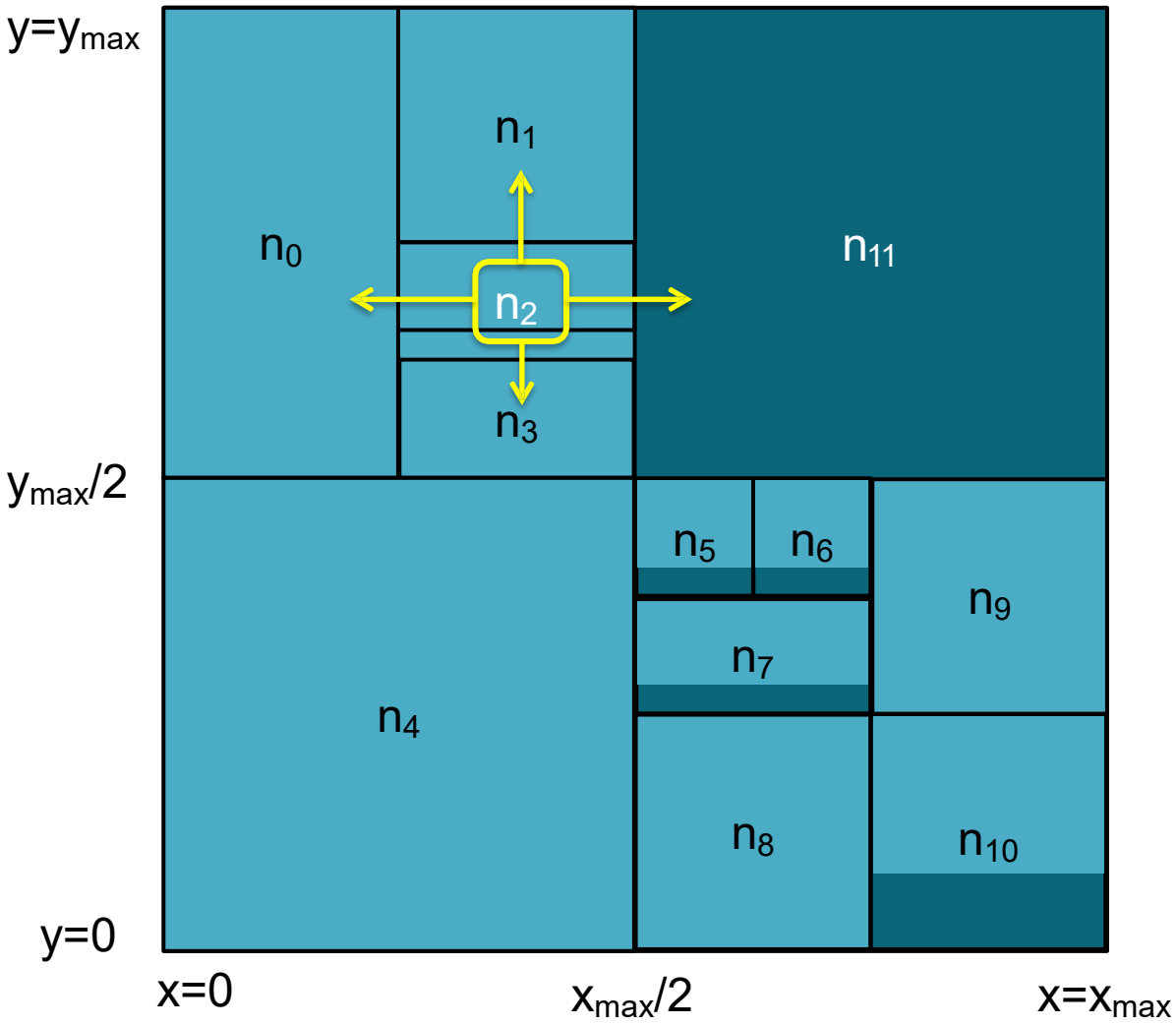
Any node can be split in two – either horizontally or vertically

Associated data has to be moved to the new node based on *hash*(*key*)

Neighbors need to be made aware of the new node

A node knows only of its neighbors

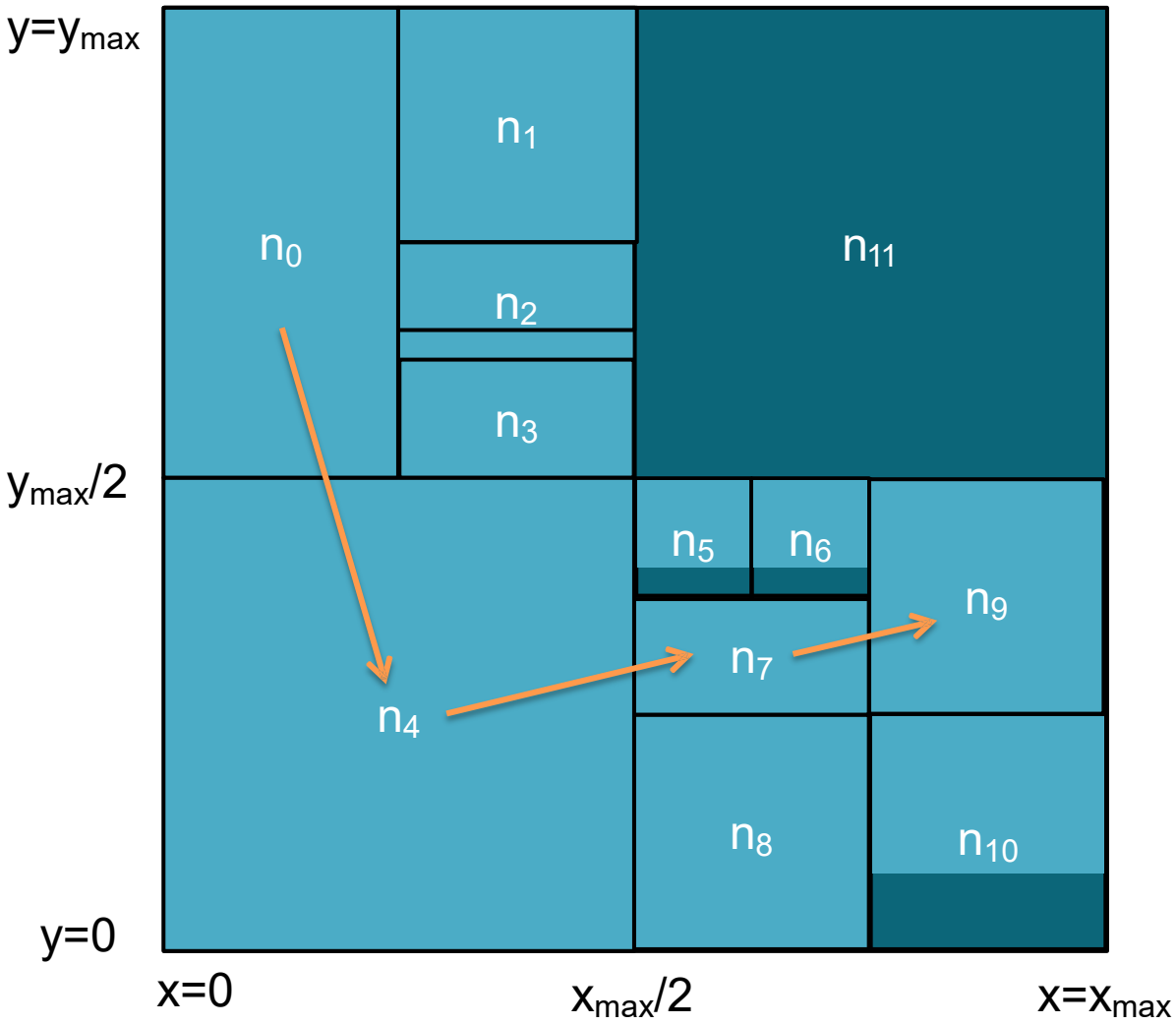
CAN neighbors



Neighbors refer to nodes that share adjacent zones in the overlay network

n_4 only needs to keep track of n_5, n_7, *or* n_8 as its right neighbor.

CAN routing



lookup(key) on a node that does not own the value

Compute $hash_x(key)$, $hash_y(key)$ and route request to a neighboring node

Ideally: route to minimize distance to destination

CAN

- Performance
  - For *n* nodes in *d* dimensions
  - # neighbors = *2d*
  - Average route for 2 dimensions = $O(\sqrt{n})$ hops

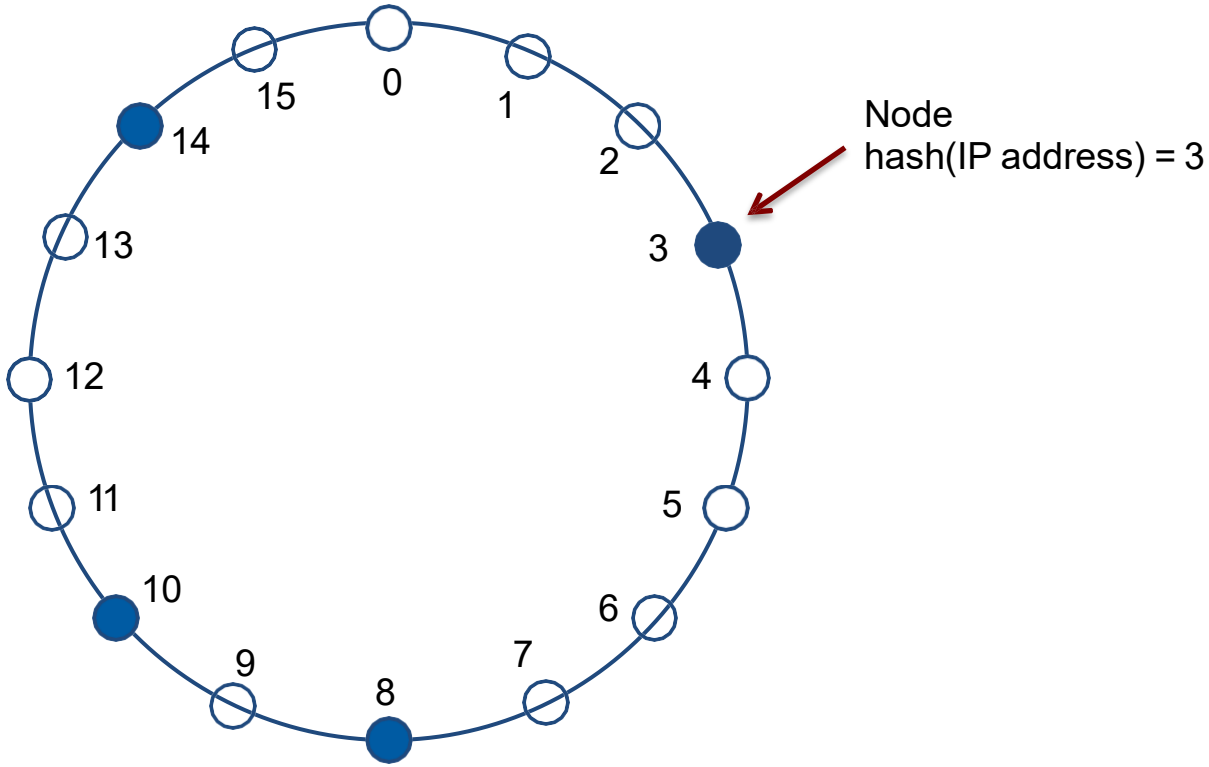
- To handle failures
  - Share knowledge of neighbor's neighbors
  - One of the node's neighbors takes over the failed zone

Distributed Hashing
Case Study

Chord

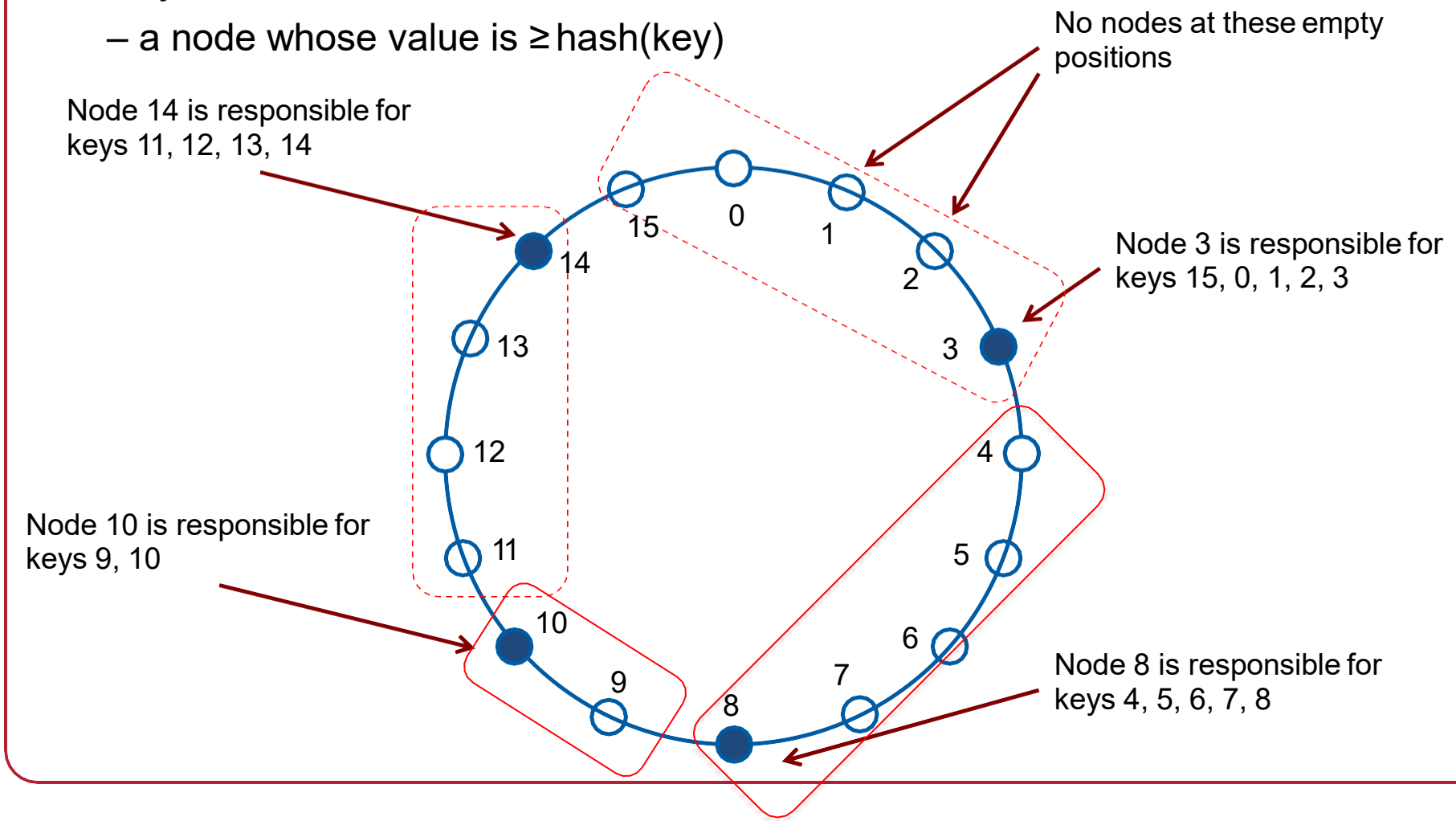
Chord & consistent hashing

- A key is hashed to an *m*-bit value: 0 … (2^m-1)
- A logical ring is constructed for the values 0 ... (2^m-1)
- Nodes are placed on the ring at *hash(IP address)*



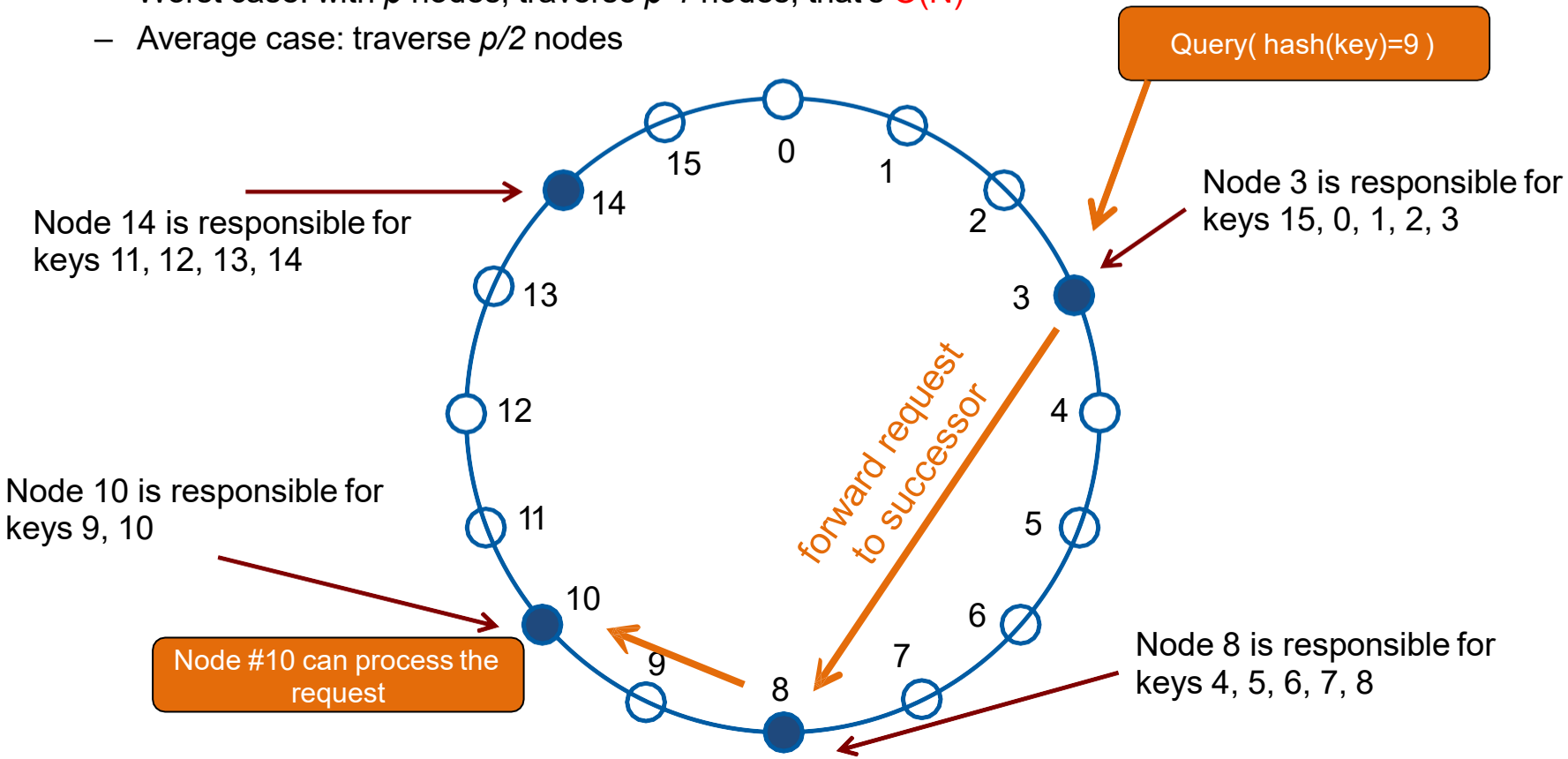
Key assignment

- Example: *n=16;* system with 4 nodes (so far)
- Key, value data is stored at a **successor**
  – a node whose value is ≥ hash(key)



Handling query requests

- Any peer can get a request (*insert* or *query*). If the *hash(key)* is not for its ranges of keys, it forwards the request to a successor.
- The process continues until the responsible node is found
  - Worst case: with *p* nodes, traverse *p-1* nodes; that's O(N)
  - Average case: traverse *p/2* nodes

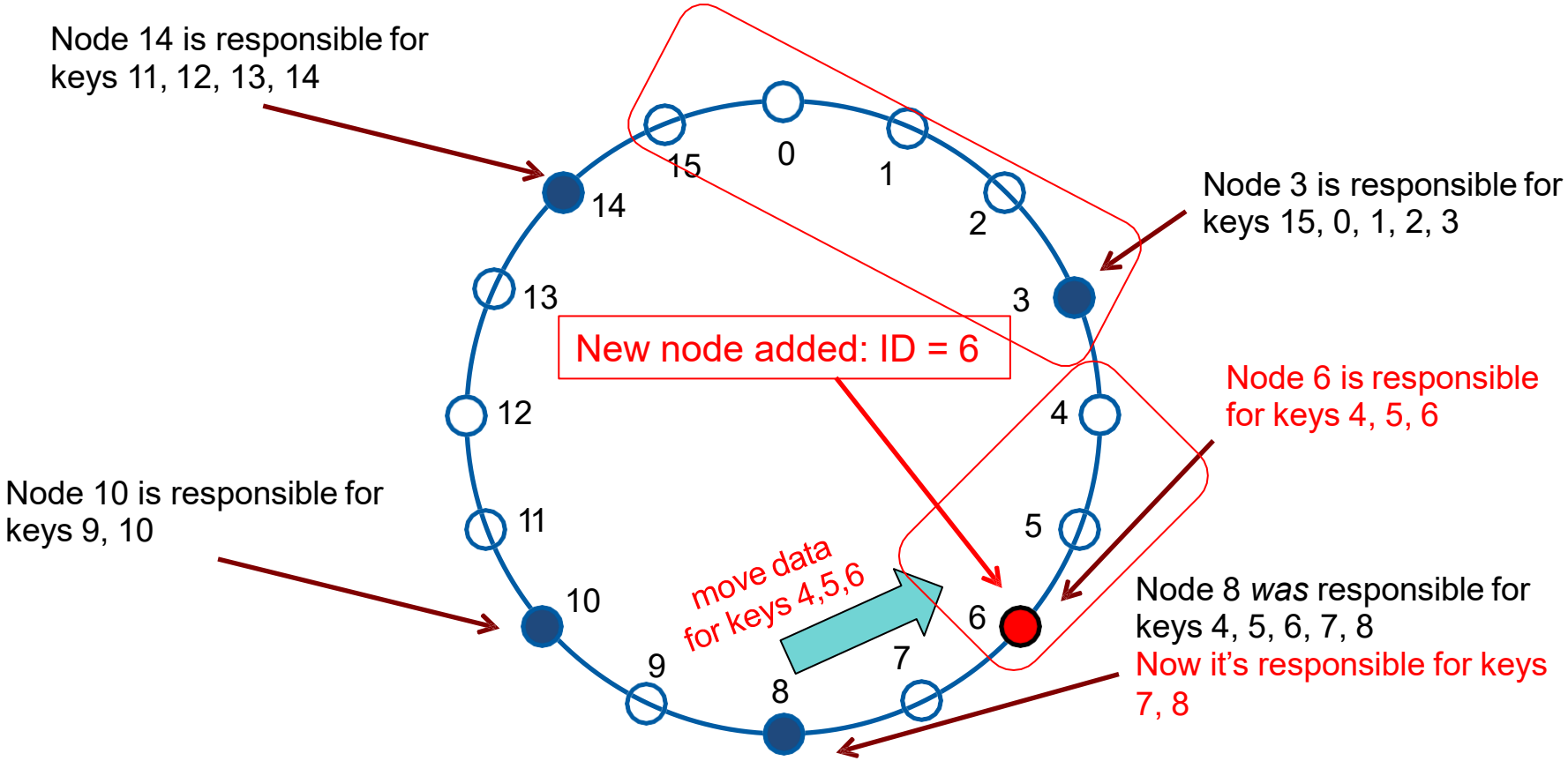


Let’s figure out three more things

1. Adding/removing nodes
2. Improving lookup time
3. Providing fault tolerance

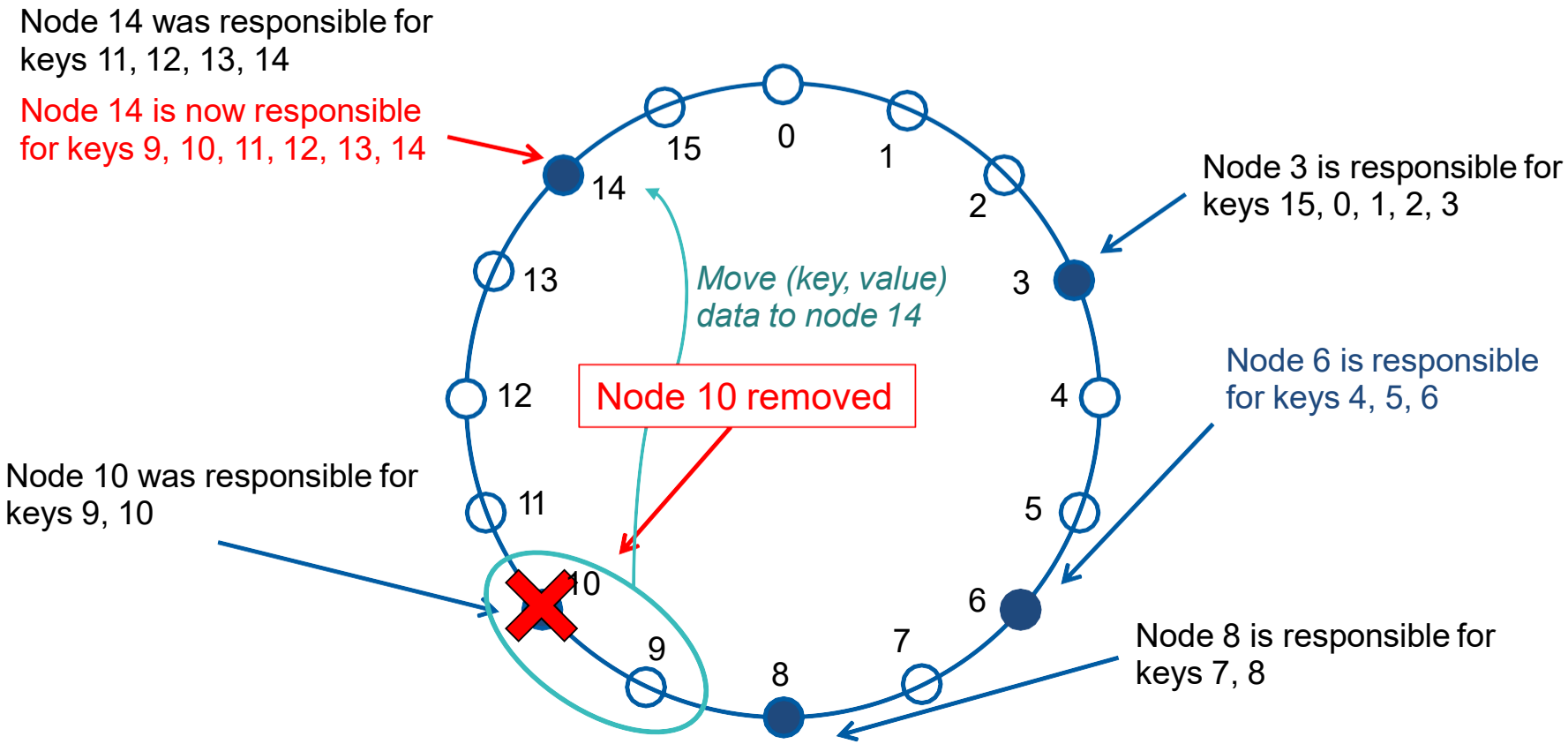
Adding a node

- Some keys that were assigned to a node's successor now get assigned to the new node
- Data for those *(key, value)* pairs must be moved to the new node



Removing a node

- Keys are reassigned to the node's successor
- Data for those *(key, value)* pairs must be moved to the successor



Fault tolerance

- Nodes might die
  - *(key, value)* data should be replicated
  - Create *R* replicas, storing each one at *R-1* successor nodes in the ring

- Need to know multiple successors
  - A node needs to know how to find its successor's successor (or more)
    - Easy if it knows all nodes!
  - When a node is back up, it needs to check with successors for updates
  - Any changes need to be propagated to all replicas

Performance

- *O(N)* lookup

- Simple approach for great performance
  - Have all nodes know about each other
  - When a peer gets a node, it searches its table of nodes for the node that owns those values
  - Gives us *O(1)* performance
  - Add/remove node operations must inform everyone
  - Maybe not a good solution if we have lots of peers (huge tables)

Finger tables

- Compromise to avoid large tables at each node
  - Use finger tables to place an upper bound on the table size
- Finger table = partial list of nodes, progressively more distant
- At each node, i^{th} entry in finger table identifies node that succeeds it by at least 2^{i-1} in the circle
  - `finger_table[0]`: immediate (1st) successor
  - `finger_table[1]`: successor after that (2nd)
  - `finger_table[2]`: 4th successor
  - `finger_table[3]`: 8th successor
  - …
- *O(log N)* nodes need to be contacted to find the node that owns a key
  … not as good as *O(1)* but way better than *O(N)*

Distributed Hashing
Case Study

Amazon Dynamo

Amazon Dynamo

- Not exposed as a web service
  - Used to power parts of Amazon Web Services and internal services
  - Highly available, key-value storage system

- In an infrastructure with millions of components, something is always failing!
  - Failure is the normal case

- A lot of services within Amazon only need primary-key access to data
  - Best seller lists, shopping carts, preferences, session management, sales rank, product catalog
  - No need for complex querying or management offered by an RDBMS
    - Full relational database is overkill: limits scale and availability
    - Still not efficient to scale or load balance RDBMS on a large scale

Core Assumptions & Design Decisions

- Two operations: **get(key)** and **put(key, data)**
  - Binary objects (data) identified by a unique key
  - Objects tend to be small (< 1MB)
- ACID gives poor availability
  - Use weaker consistency (C) for higher availability.
- Apps should be able to configure Dynamo for desired latency & throughput
  - Balance performance, cost, availability, durability guarantees.
- At least 99.9% of read/write operations must be performed within a few hundred milliseconds:
  - Avoid routing requests through multiple nodes
- Dynamo can be thought of as a zero-hop DHT

Core Assumptions & Design Decisions

- Incremental scalability
  - System should be able to grow by adding a storage host (node) at a time
- Symmetry
  - Every node has the same set of responsibilities
- Decentralization
  - Favor decentralized techniques over central coordinators
- Heterogeneity (mix of slow and fast systems)
  - Workload partitioning should be proportional to capabilities of servers

Consistency & Availability

- Strong consistency & high availability cannot be achieved simultaneously

- Optimistic replication techniques – eventually consistent model
  - propagate changes to replicas in the background
  - can lead to conflicting changes that have to be detected & resolved

- When do you resolve conflicts?
  - **During writes**: traditional approach – reject write if cannot reach all (or majority) of replicas – ***but don't deal with conflicts***
  - **Resolve conflicts during reads**: Dynamo approach
    - Design for an "always writable" data store - highly available
    - read/write operations can continue even during network partitions
    - Rejecting customer updates won't be a good experience
      - A customer should always be able to add or remove items in a shopping cart

Reads & Writes

Two operations:

- get(key) returns
  1. object or list of objects with conflicting versions
  2. context (resultant version per object)
- put(key, context, value)
  - stores replicas
  - *context*: ignored by the application but includes version of object
  - key is hashed with MD5 to create a 128-bit identifier that is used to determine the storage nodes that serve the key

hash(key) identifies node

Partitioning the data

- Break up database into chunks distributed over all nodes
  - Key to scalability

- Relies on consistent hashing
  - K/n keys need to be remapped, K = # keys, n = # slots

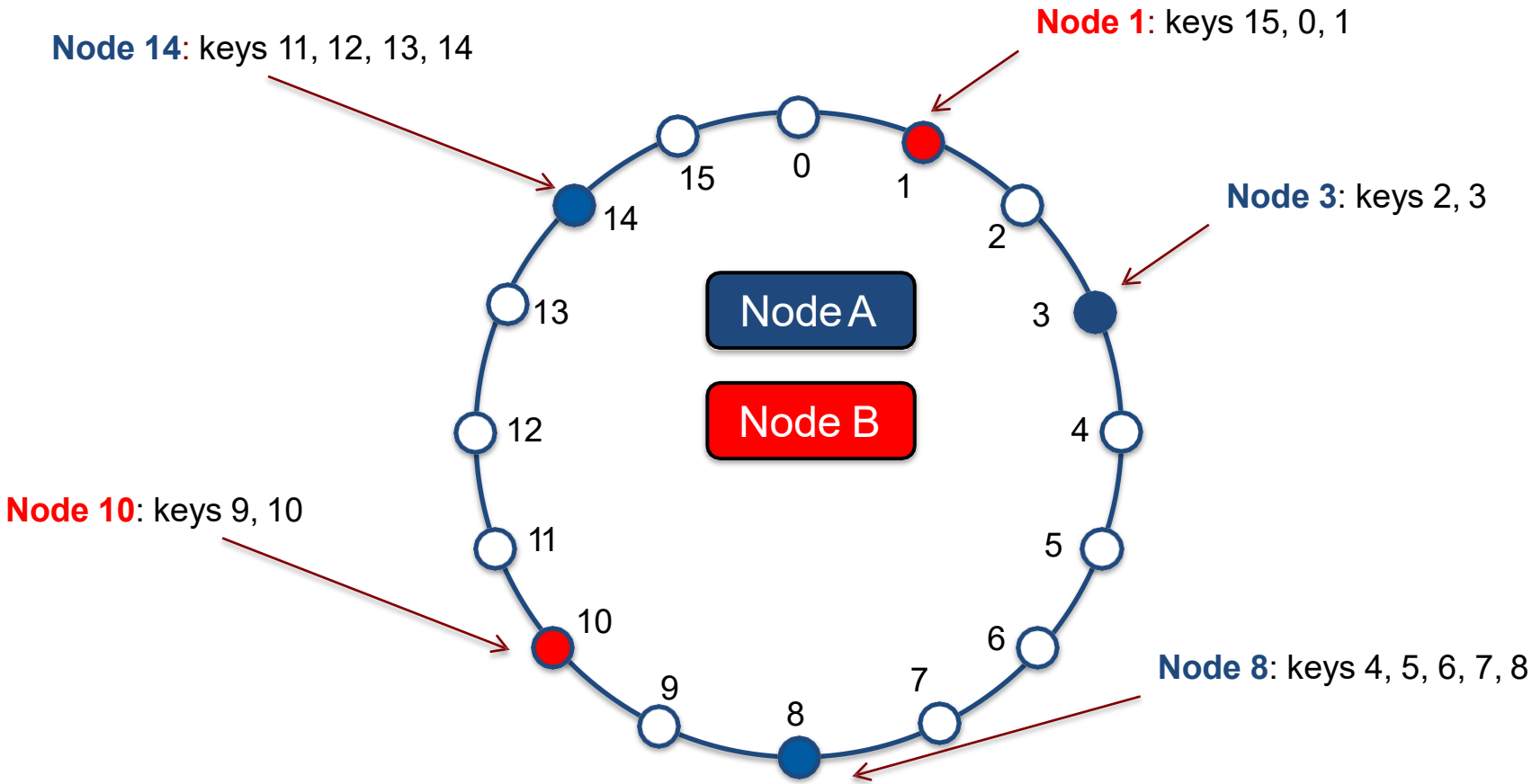
- Logical ring of nodes: just like Chord
  - Each node assigned a random value in the hash space: position in ring
  - Responsible for all hash values between its value and predecessor's value
  - Hash(key); then walk ring clockwise to find first node with position>hash
  - Adding/removing nodes affects only immediate neighbors

Partitioning: virtual nodes

- A node is assigned to multiple points in the ring
- Each point is a “virtual node”

Dynamo virtual nodes

- A physical node holds contents of multiple virtual nodes
- In this example: 2 physical nodes, 5 virtual nodes



Partitioning: virtual nodes

Advantage: balanced load distribution

- If a node becomes unavailable, load is evenly dispersed among available nodes
- If a node is added, it accepts an equivalent amount of load from other available nodes
- # of virtual nodes per system can be based on the capacity of that node
  - Makes it easy to support changing technology and addition of new, faster systems

Replication

- Data replicated on *N* hosts (*N* is configurable)
  - Key is assigned a coordinator node (via hashing) = main node
  - Coordinator is in charge of replication

- Coordinator replicates keys at the *N-1* clockwise successor nodes in the ring

Dynamo Replication

Coordinator replicates keys at the *N-1* clockwise successor nodes in the ring

